

(19) **United States**
(12) **Patent Application Publication** (10) **Pub. No.: US 2025/0279925 A1**
Shribman et al. (43) **Pub. Date: Sep. 4, 2025**

(54) **SYSTEM PROVIDING FASTER AND MORE EFFICIENT DATA COMMUNICATION**

(71) Applicant: **BRIGHT DATA LTD.**, Netanya (IL)

(72) Inventors: **Derry Shribman**, Tel Aviv (IL); **Ofer Vilenski**, Moshav Hadar Am (IL)

(21) Appl. No.: **19/179,272**

(22) Filed: **Apr. 15, 2025**

Related U.S. Application Data

- (60) Continuation of application No. 18/390,397, filed on Dec. 20, 2023, now Pat. No. 12,323,287, which is a continuation of application No. 17/714,475, filed on Apr. 6, 2022, now Pat. No. 11,888,922, which is a continuation of application No. 17/146,728, filed on Jan. 12, 2021, now Pat. No. 11,303,734, which is a continuation of application No. 17/019,268, filed on Sep. 13, 2020, now Pat. No. 10,931,792, which is a continuation of application No. 16/910,863, filed on Jun. 24, 2020, now Pat. No. 10,805,429, which is a continuation of application No. 16/600,504, filed on Oct. 13, 2019, now Pat. No. 11,044,341, which is a continuation of application No. 16/278,105, filed on Feb. 17, 2019, now Pat. No. 10,469,628, which is a continuation of application No. 15/957,950, filed on Apr. 20, 2018, now Pat. No. 10,225,374, which is a continuation of application No. 14/025,109, filed on Sep. 12, 2013, now Pat. No. 10,069,936, which is a division of application No. 12/836,059, filed on Jul. 14, 2010, now Pat. No. 8,560,604.
- (60) Provisional application No. 61/249,624, filed on Oct. 8, 2009.

Publication Classification

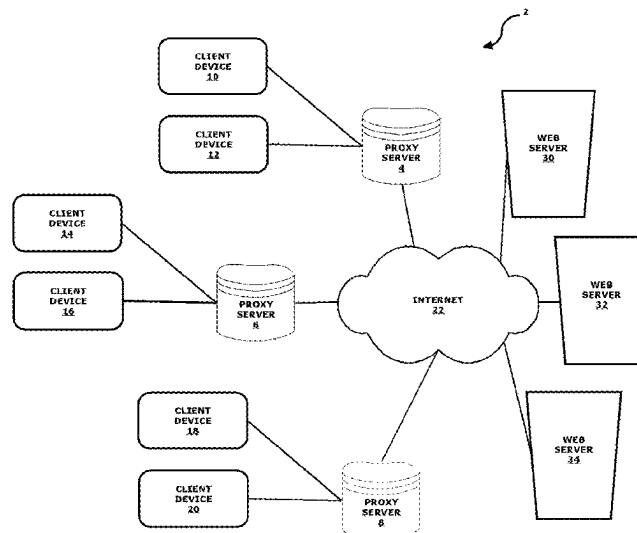
- (51) **Int. Cl.**
H04L 41/046 (2022.01)
H04L 67/01 (2022.01)
H04L 67/02 (2022.01)

H04L 67/1001 (2022.01)
H04L 67/1023 (2022.01)
H04L 67/1061 (2022.01)
H04L 67/1074 (2022.01)
H04L 67/142 (2022.01)
H04L 67/50 (2022.01)
H04L 67/563 (2022.01)
H04L 67/564 (2022.01)

(52) **U.S. Cl.**
CPC **H04L 41/046** (2013.01); **H04L 67/01** (2022.05); **H04L 67/1001** (2022.05); **H04L 67/1023** (2013.01); **H04L 67/1063** (2013.01); **H04L 67/108** (2013.01); **H04L 67/142** (2013.01); **H04L 67/535** (2022.05); **H04L 67/563** (2022.05); **H04L 67/564** (2022.05); **H04L 67/02** (2013.01)

(57) **ABSTRACT**

A system designed for increasing network communication speed for users, while lowering network congestion for content owners and ISPs. The system employs network elements including an acceleration server, clients, agents, and peers, where communication requests generated by applications are intercepted by the client on the same machine. The IP address of the server in the communication request is transmitted to the acceleration server, which provides a list of agents to use for this IP address. The communication request is sent to the agents. One or more of the agents respond with a list of peers that have previously seen some or all of the content which is the response to this request (after checking whether this data is still valid). The client then downloads the data from these peers in parts and in parallel, thereby speeding up the Web transfer, releasing congestion from the Web by fetching the information from multiple sources, and relieving traffic from Web servers by offloading the data transfers from them to nearby peers.



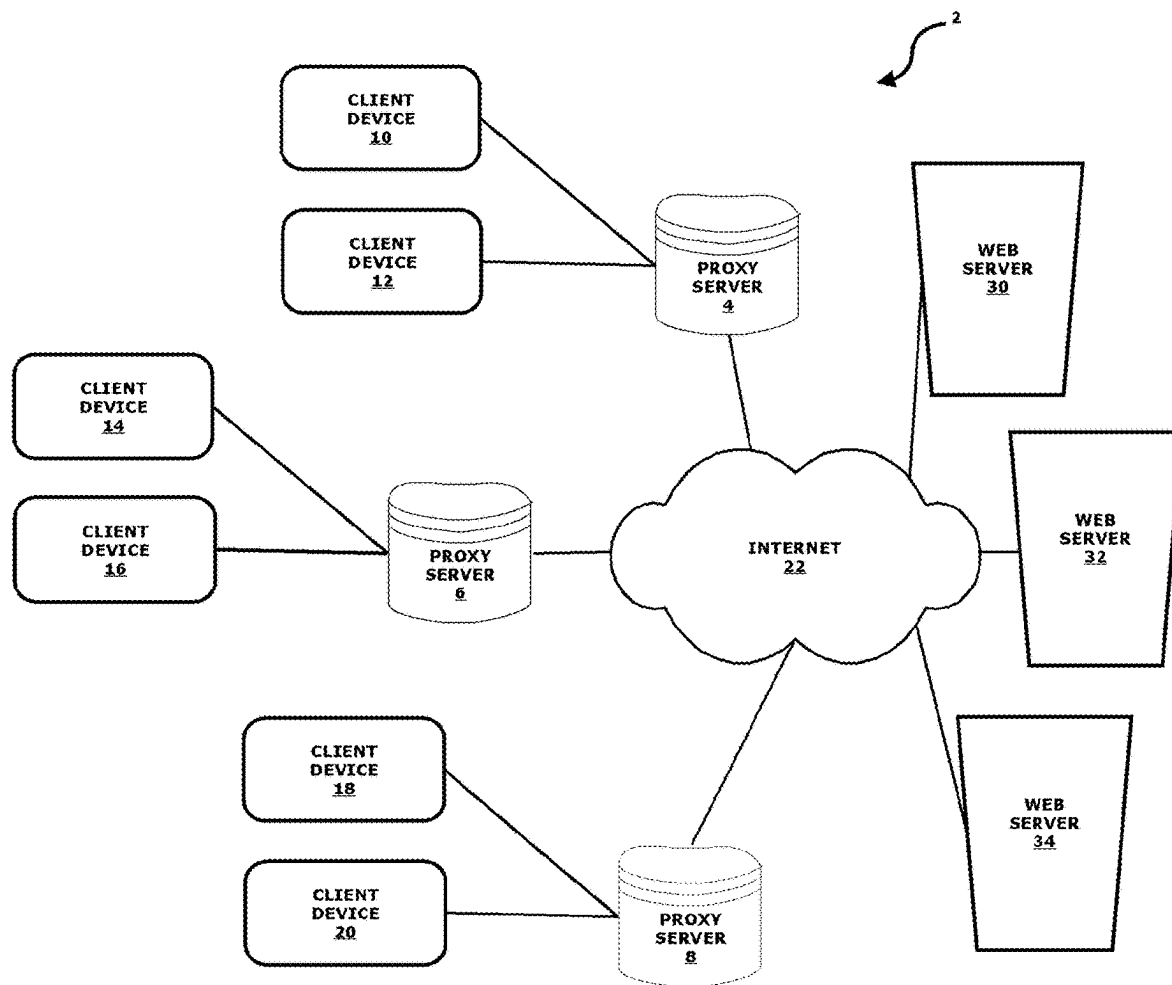


FIG. 1

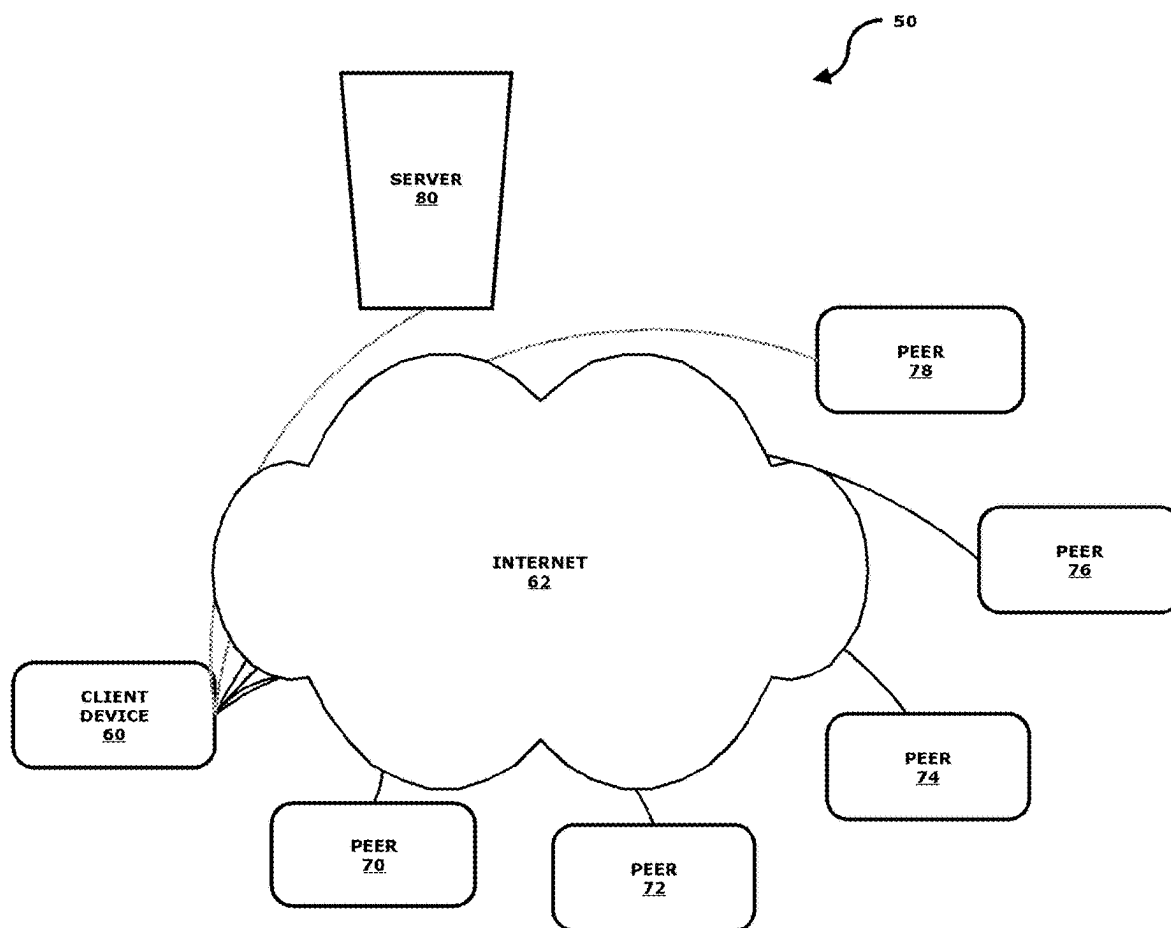


FIG. 2

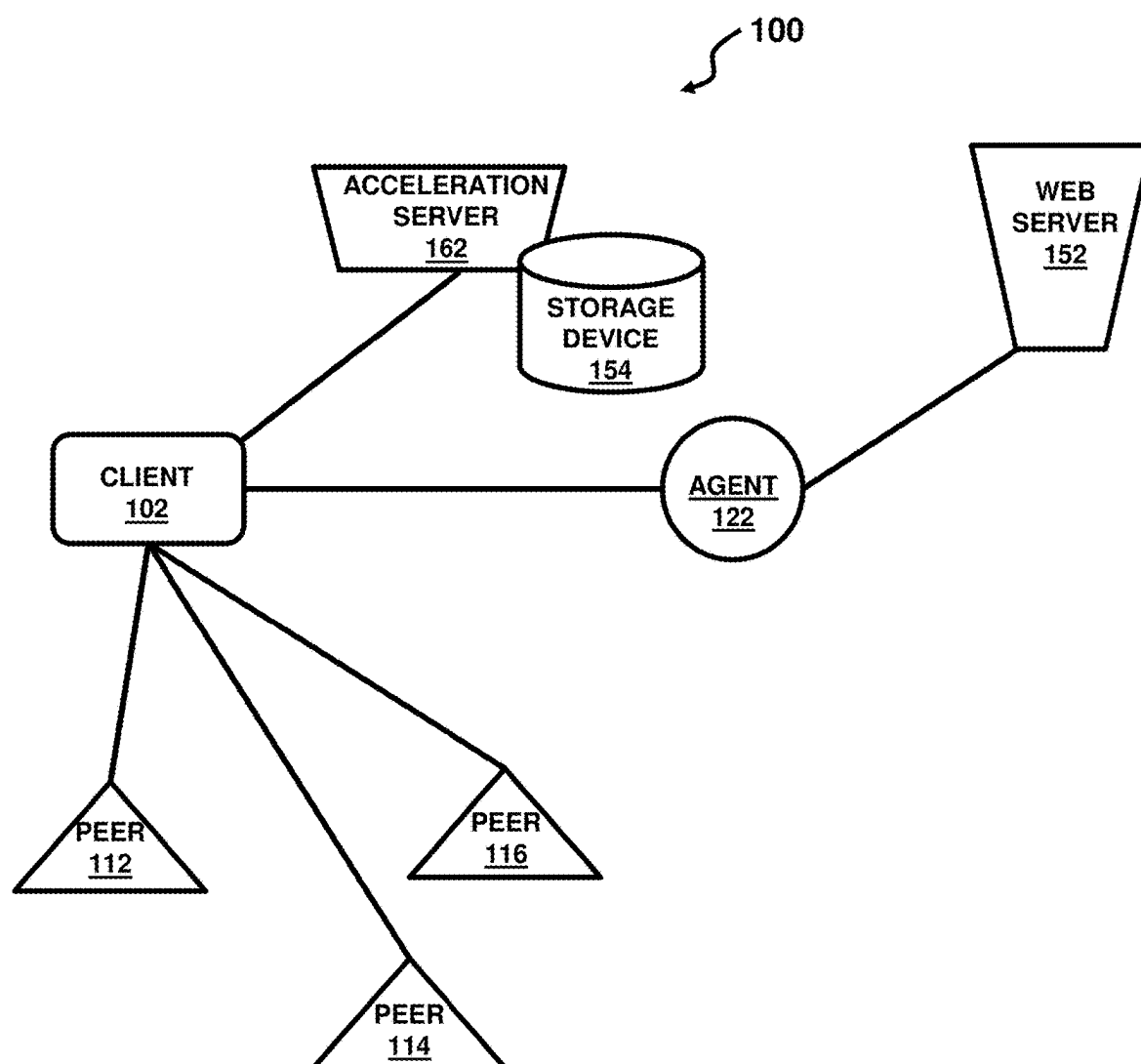


FIG. 3

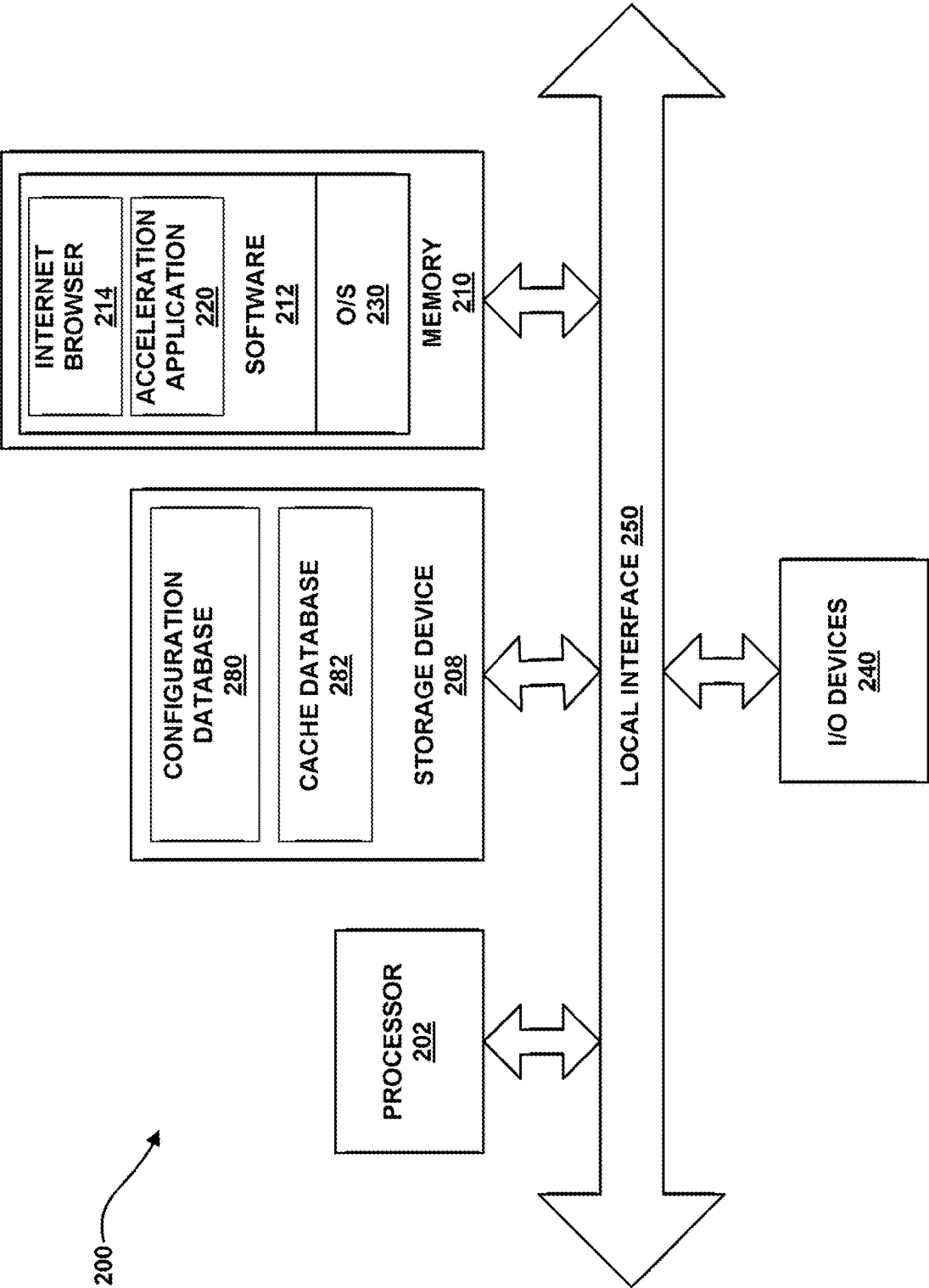


FIG. 4

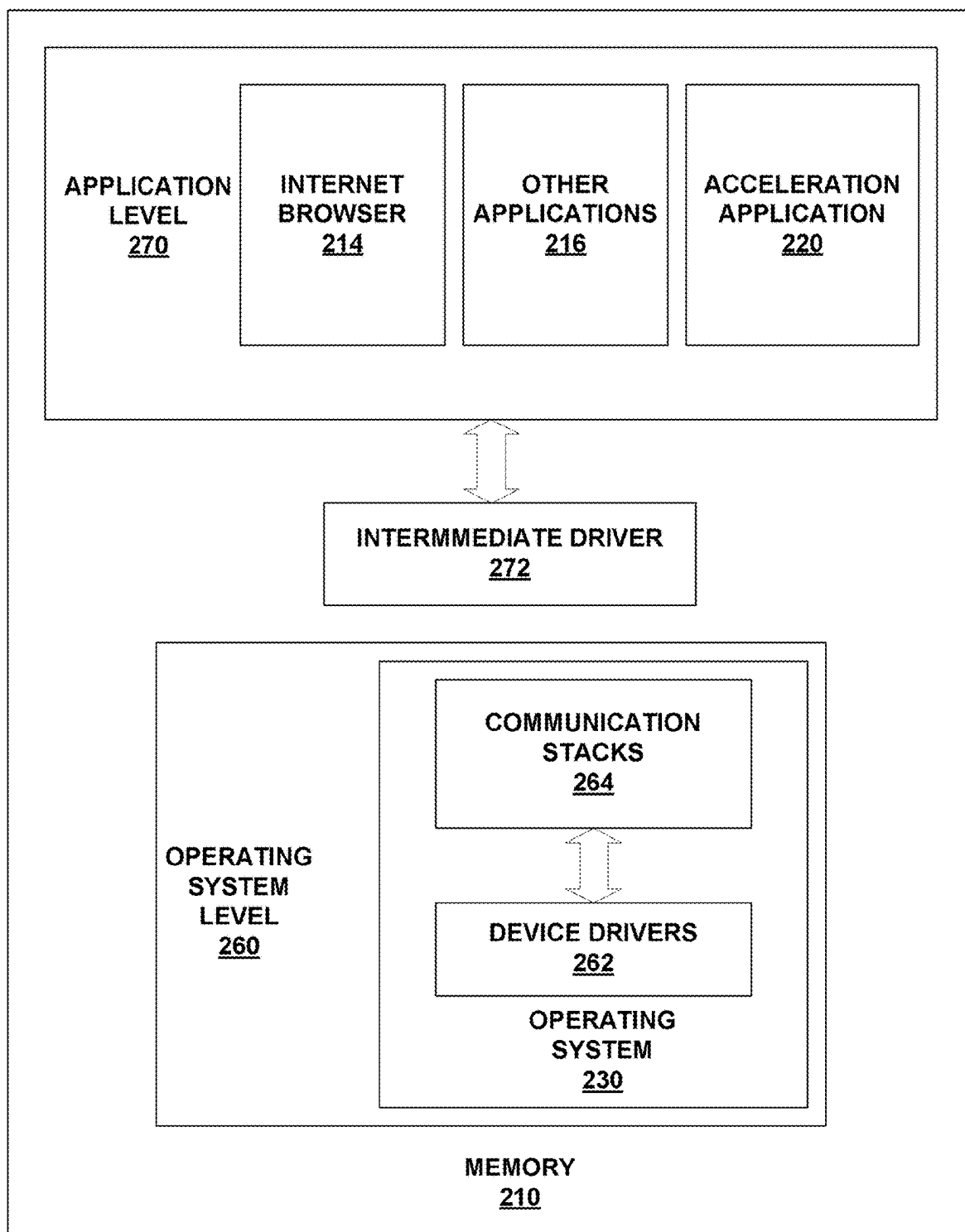


FIG. 5

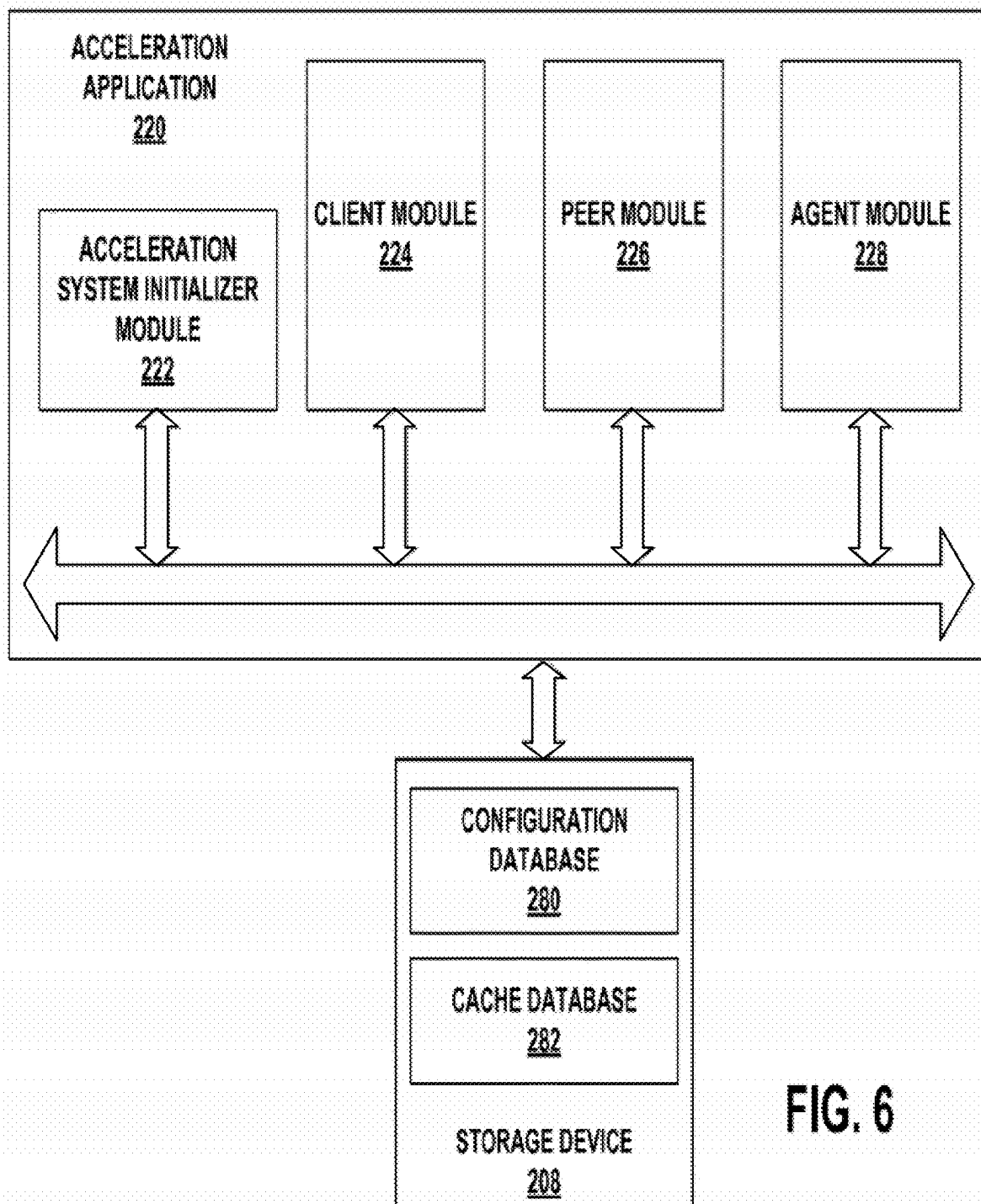
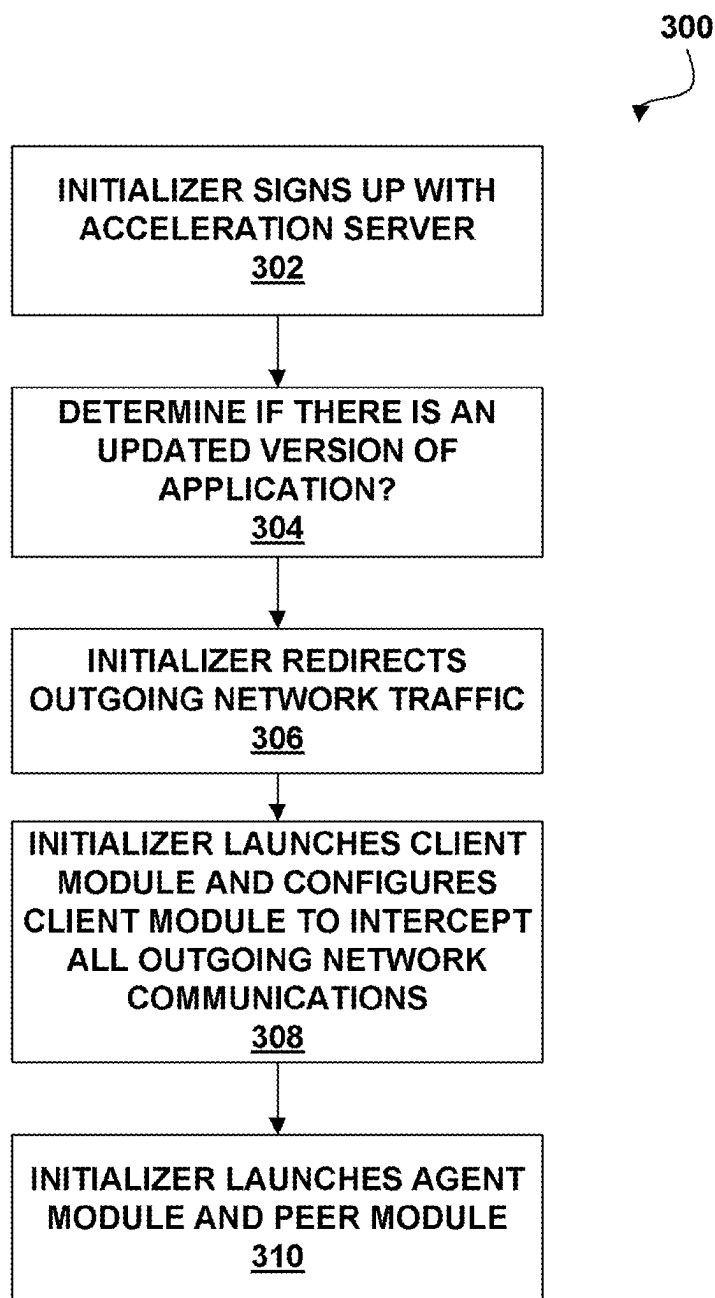


FIG. 6

162

ACCELERATION DATABASE 164									
166	AGENT IP A	ONLINE/OFFLINE							
	>>>INDEXED BY: AGENT IP ADDRESS								
CACHE DATABASE 282									
286	LIST OF URLS:								
288	URL 1								
		290	URL						
		292	URL HTTP HEADERS						
		294	LAST CHECKED ON SERVER						
		296	LAST CHANGED ON SERVER						
		298	LIST OF CHUNKS FOR THIS URL:						
			300	CHUNK 1					
				302	CHUNK CHECKSUM				
				304	CHUNK DATA				
				306	LIST OF PEERS:				
					308	PEER 1			
							310	PEER 1 IP ADDRESS	
							312	PEER 2 CONNECTION STATUS	

FIG. 7

**FIG. 8**

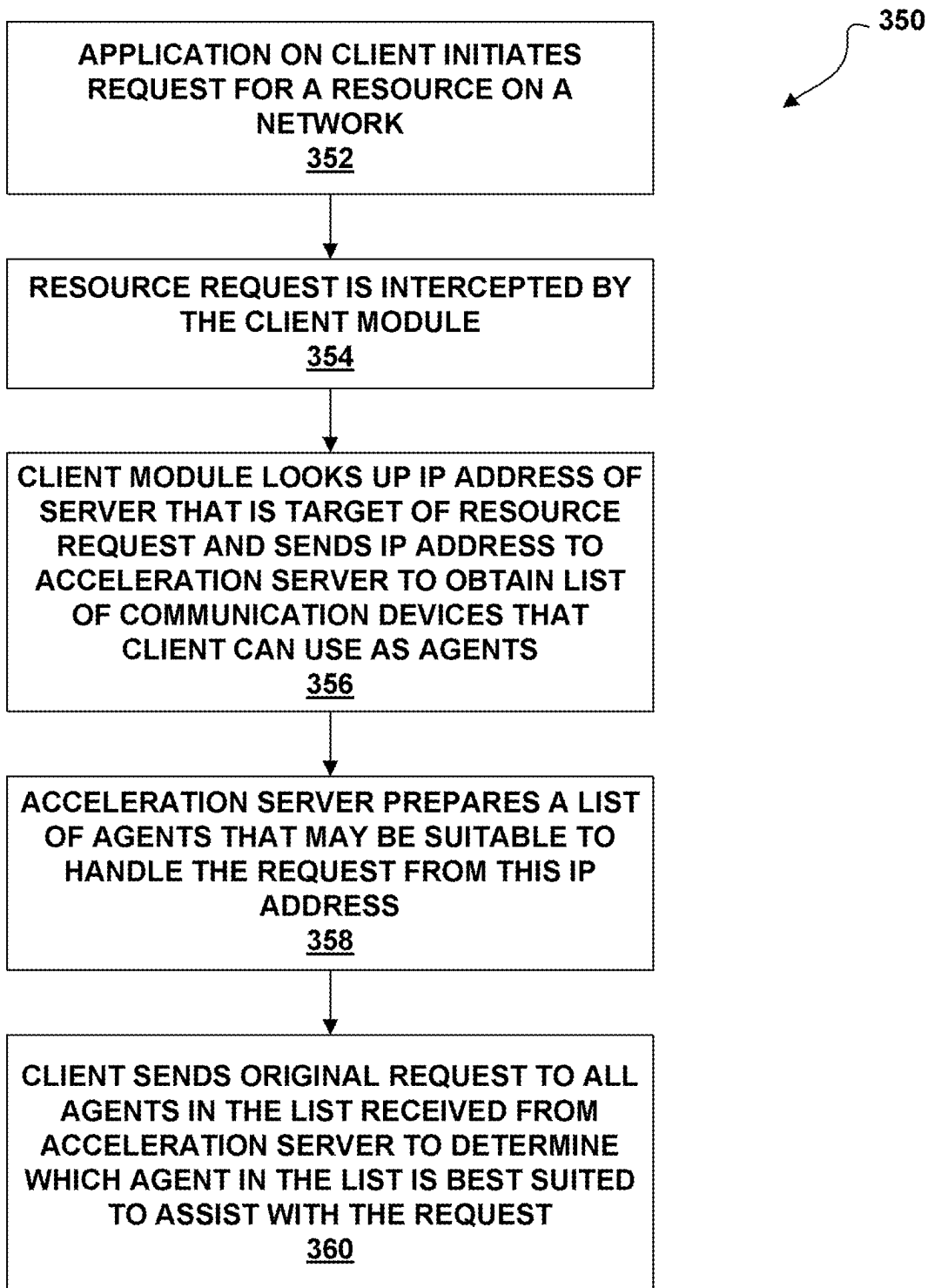


FIG. 9

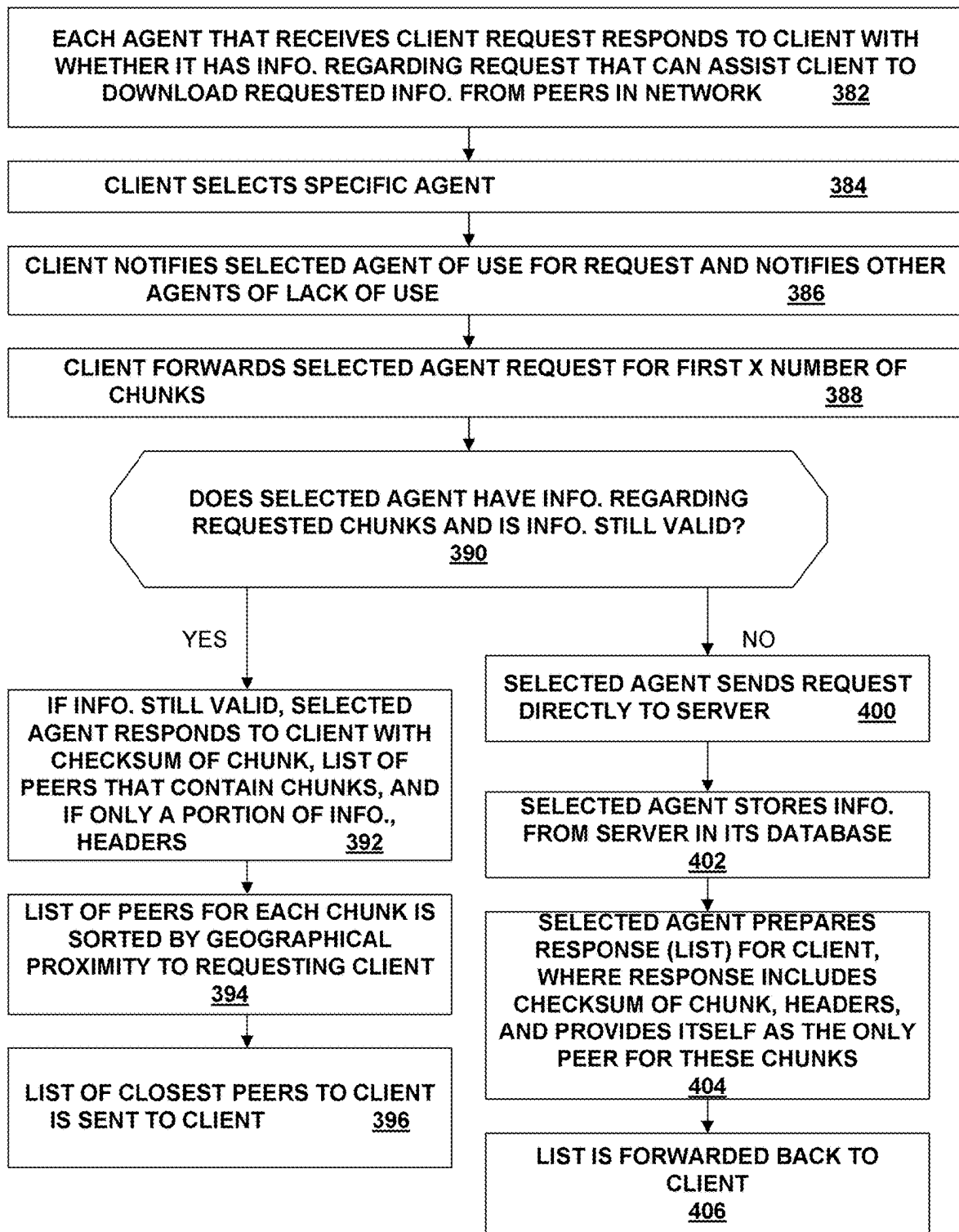


FIG. 10

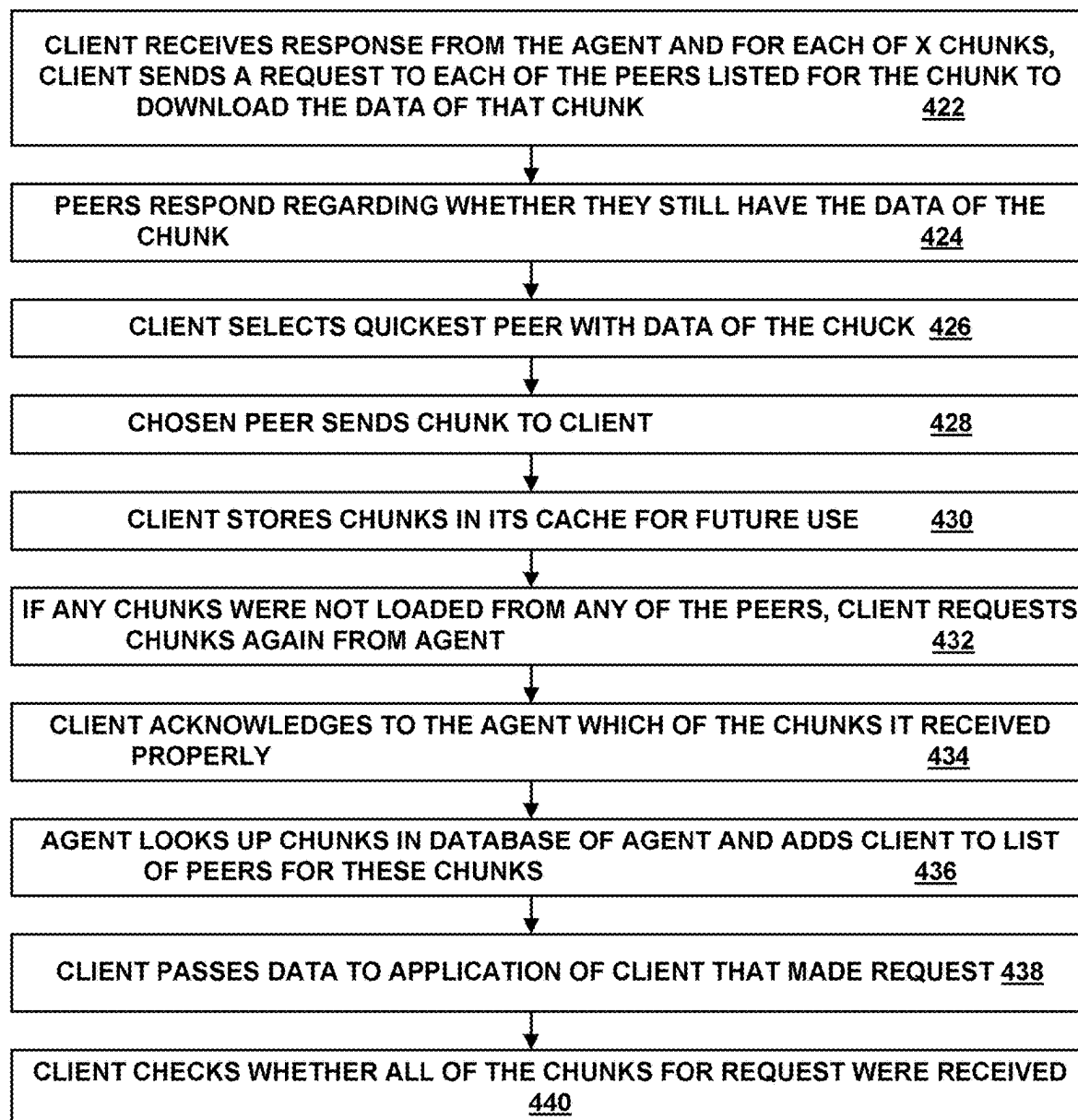


FIG. 11

420

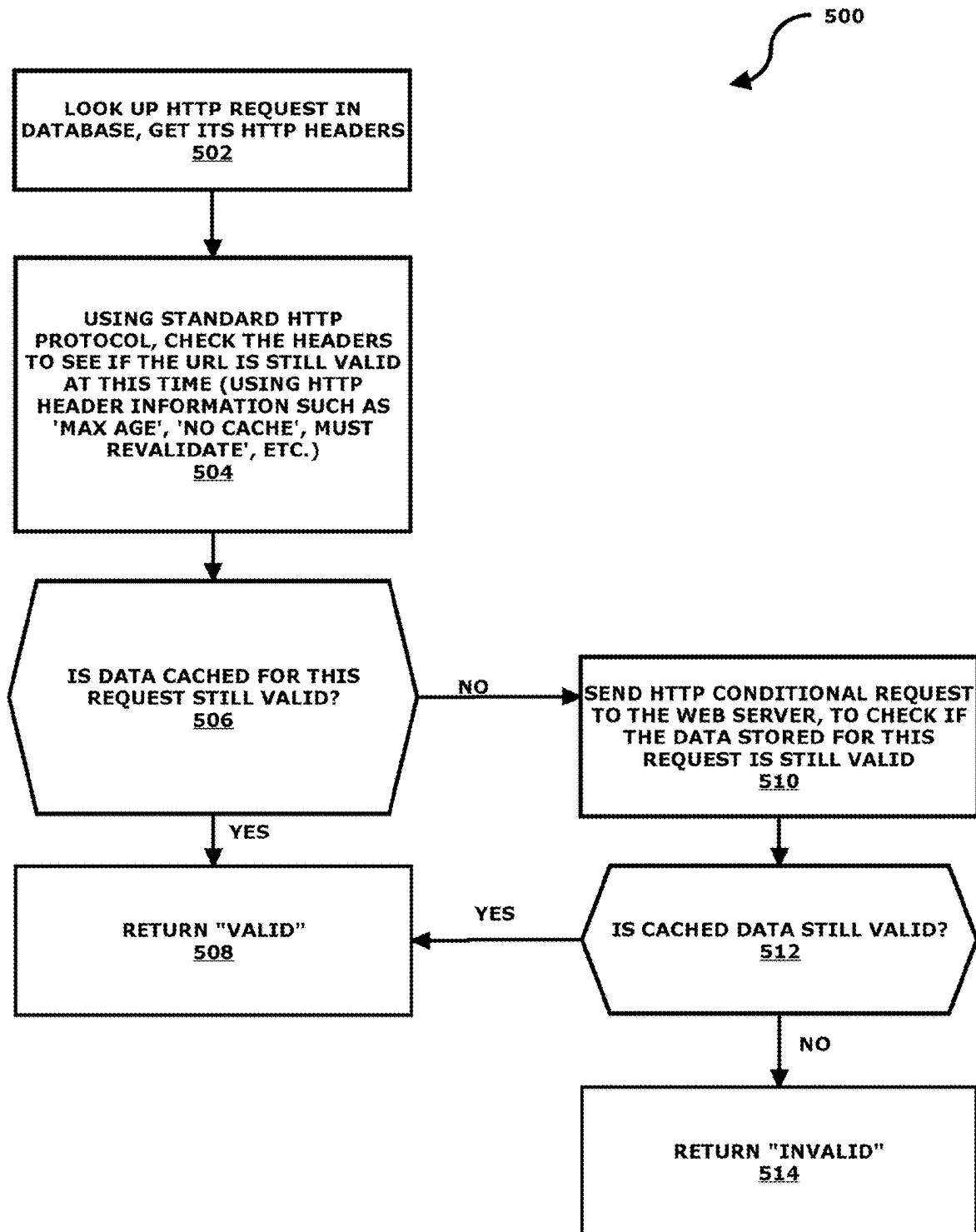


FIG. 12

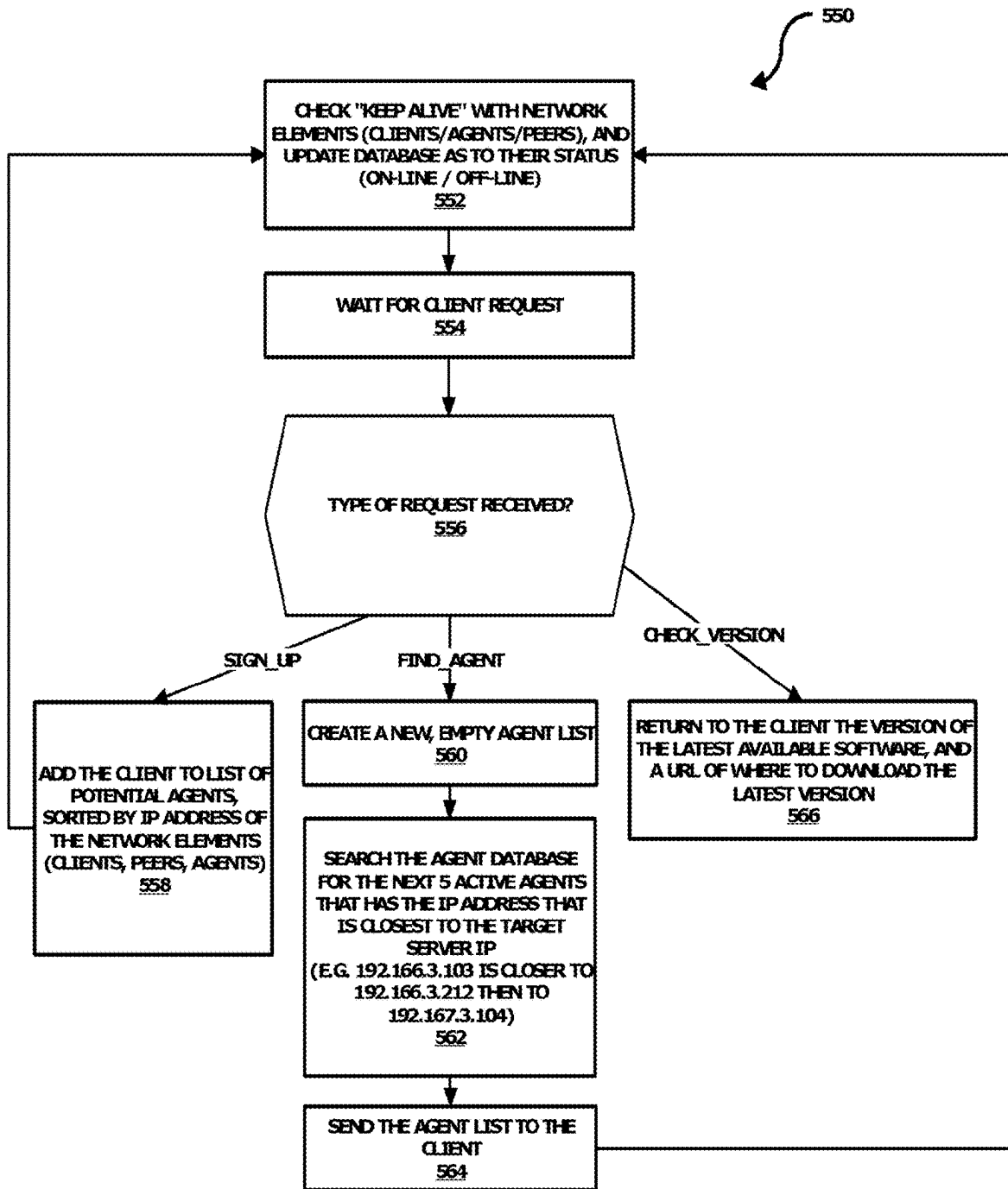


FIG 13

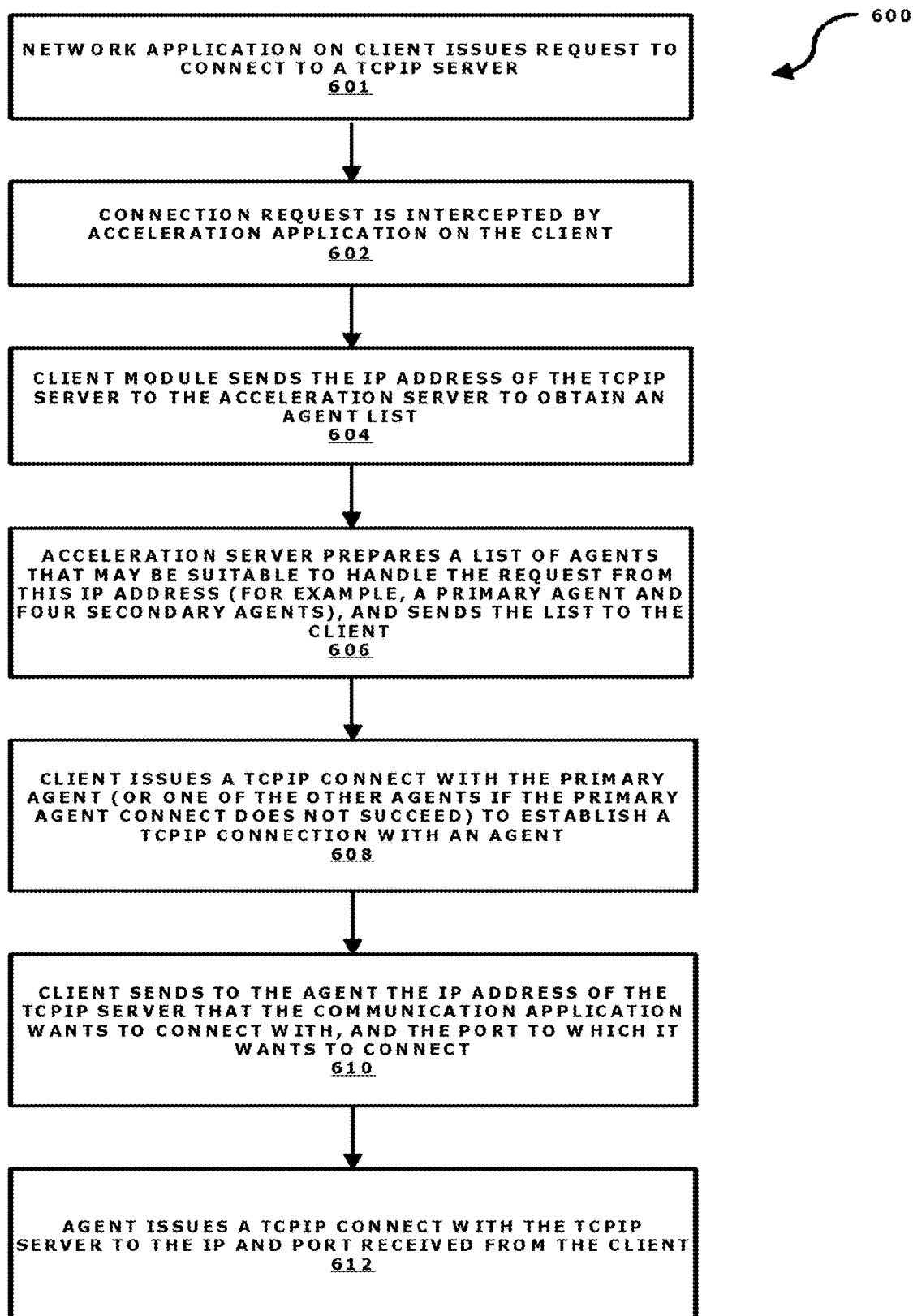


FIG. 14

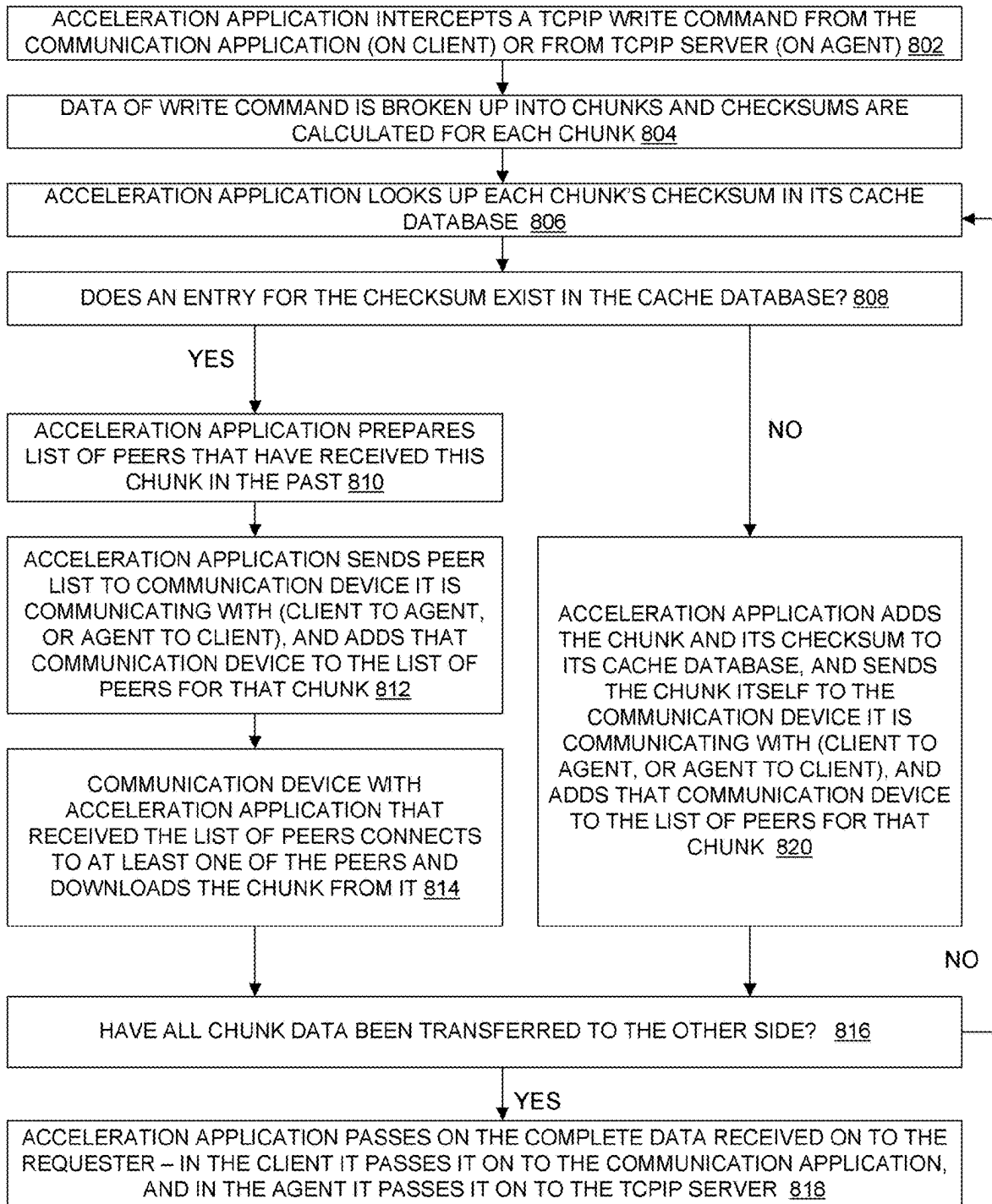


FIG. 15

800

SYSTEM PROVIDING FASTER AND MORE EFFICIENT DATA COMMUNICATION

RELATED APPLICATIONS

[0001] The present application is a continuation application of U.S. application Ser. No. 18/390,397, filed on Dec. 20, 2023, which is a continuation application of U.S. application Ser. No. 17/714,475, filed on Apr. 6, 2022 (now U.S. Pat. No. 11,888,922), which is a continuation application of U.S. application Ser. No. 17/146,728, filed on Jan. 12, 2021 (now U.S. Pat. No. 11,303,734), which is a continuation application of U.S. application Ser. No. 17/019,268, filed on Sep. 13, 2020 (now U.S. Pat. No. 10,931,792), which is a continuation application of U.S. application Ser. No. 16/910,863, filed on Jun. 24, 2020 (now U.S. Pat. No. 10,805,429), which is a continuation application of U.S. application Ser. No. 16/600,504, filed on Oct. 13, 2019 (now U.S. Pat. No. 11,044,341), which is a continuation application of U.S. application Ser. No. 16/278,105, filed on Feb. 17, 2019 (now U.S. Pat. No. 10,469,628), which is a continuation application of U.S. application Ser. No. 15/957,950, filed on Apr. 20, 2018 (now U.S. Pat. No. 10,225,374), which is a continuation application of U.S. patent application Ser. No. 14/025,109, filed Sep. 12, 2013 (now U.S. Pat. No. 10,069,936), which is a divisional application of U.S. patent application entitled “SYSTEM AND METHOD FOR PROVIDING FASTER AND MORE EFFICIENT DATA COMMUNICATION” having Ser. No. 12/836,059, filed Jul. 14, 2010 and issued as U.S. Pat. No. 8,560,604 on Oct. 15, 2013, and claims priority to U.S. provisional patent application entitled “SYSTEM AND METHOD FOR REDUCING INTERNET CONGESTION,” having Ser. No. 61/249,624, filed Oct. 8, 2009, which are all hereby incorporated herein by reference in their entirety.

TECHNICAL FIELD

[0002] The present invention is related to Internet communication, and more particularly, to improving data communication speed and bandwidth efficiency on the Internet.

BACKGROUND

[0003] There are several trends in network and Internet usage, which tremendously increase the bandwidth that is being used on the Internet. One such trend is that more and more video is being viewed on demand on the Internet. Such viewing includes the viewing of both large and short video clips. In addition, regular shows and full-featured films may be viewed on the Internet. Another trend that is increasing the traffic on the Internet is that Web sites (such as shopping portals, news portals, and social networks) are becoming global, meaning that the Web sites are serving people in many diverse places on the globe, and thus the data is traversing over longer stretches of the Internet, increasing the congestion.

[0004] The increase in bandwidth consumption has created several major problems, a few of which are described below:

[0005] The problem for users—the current Internet bandwidth is not sufficient, and thus the effective ‘speed’ experienced by users is slow;

[0006] The problem for content owners—the tremendous amount of data being viewed by users is costing large amounts of money in hosting and bandwidth costs; and

[0007] The problem for Internet Service Providers (ISPs) —the growth in Internet traffic is requiring the ISPs to increase the infrastructure costs (communication lines, routers, etc.) at tremendous financial expense.

[0008] The need for a new method of data transfer that is fast for the consumer, cheap for the content distributor and does not require infrastructure investment for ISPs, has become a major issue which is yet unsolved.

[0009] There have been many attempts at making the Internet faster for the consumer and cheaper for the broadcaster. Each such attempt is lacking in some aspect to become a widespread, practical solution, or is a partial solution in that it solves only a subset of the major problems associated with the increase in Internet traffic. Most of the previous solutions require billions of dollars in capital investment for a comprehensive solution. Many of these attempts are lacking in that much of the content on the Internet has become dynamically created per the user and the session of the user (this is what used to be called the “Web2.0” trend). This may be seen on the Amazon Web site and the Salesforce Web site, for example, where most of the page views on these Web sites is tailored to the viewer, and is thus different for any two viewers. This dynamic information makes it impossible for most of the solutions offered to date to store the content and provide it to others seeking similar content.

[0010] One solution that has been in use is called a “proxy”. FIG. 1 is a schematic diagram providing an example of use of a proxy within a network 2. A proxy, or proxy server 4, 6, 8 is a device that is placed between one or more clients, illustrated in FIG. 1 as client devices 10, 12, 14, 16, 18, 20, that request data, via the Internet 22, and a Web server or Web servers 30, 32, 34 from which they are requesting the data. The proxy server 4, 6, 8 requests the data from the Web servers 30, 32, 34 on their behalf, and caches the responses from the Web servers 30, 32, 34, to provide to other client devices that make similar requests. If the proxy server 4, 6, 8 is geographically close enough to the client devices 10, 12, 14, 16, 18, 20, and if the storage and bandwidth of the proxy server 4, 6, 8 are large enough, the proxy server 4, 6, 8 will speed up the requests for the client devices 10, 12, 14, 16, 18, 20 that it is serving.

[0011] It should be noted, however, that to provide a comprehensive solution for Internet surfing, the proxy servers of FIG. 1 would need to be deployed at every point around the world where the Internet is being consumed, and the storage size of the proxy servers at each location would need to be near the size of all the data stored anywhere on the Internet. The abovementioned would lead to massive costs that are impractical. In addition, these proxy solutions cannot deal well with dynamic data that is prevalent now on the Web.

[0012] There have been commercial companies, such as Akamai, that have deployed such proxies locally around the world, and that are serving a select small group of sites on the Internet. If all sites on the Web were to be solved with such a solution, the capital investment would be in the range of billions of dollars. In addition, this type of solution does not handle dynamic content.

[0013] To create large distribution systems without the large hardware costs involved with a proxy solution, “peer-to-peer file sharing” solutions have been introduced, such as, for example, BitTorrent. FIG. 2 is a schematic diagram providing an example of a peer-to-peer file transfer network 50. In the network 50, files are stored on computers of consumers, referred to herein as client devices 60. Each consumer can serve up data to other consumers, via the Internet 62, thus taking the load of serving off of the distributors and saving them the associated costs, and providing the consumer multiple points from which to download the data, referred to herein as peers 70, 72, 74, 76, 78, thus increasing the speed of the download. However, each such peer-to-peer solution must have some sort of index by which to find the required data. In typical peer-to-peer file sharing systems, because the index is on a server 80, or distributed among several servers, the number of files available in the system is not very large (otherwise, the server costs would be very large, or the lookup time would be very long).

[0014] The peer-to-peer file sharing solution is acceptable in file sharing systems, because there are not that many media files that are of interest to the mass (probably in the order of magnitude of millions of movies and songs that are of interest). Storing and maintaining an index of millions of entries is practical technically and economically. However, if this system were to be used to serve the hundreds of billions of files that are available on the Internet of today, the cost of storing and maintaining such an index would be again in the billions of dollars. In addition, these types of peer-to-peer file sharing systems are not able to deal with dynamic HTTP data.

[0015] In conclusion, a system does not exist that enables fast transmission of most of the data on the Internet, that does not incur tremendous costs, and/or that provides only a very partial solution to the problem of Internet traffic congestion. Thus, a heretofore unaddressed need exists in the industry to address the aforementioned deficiencies and inadequacies.

SUMMARY

[0016] The present system and method provides for faster and more efficient data communication within a communication network. Briefly described, in architecture, one embodiment of the system, among others, can be implemented as follows. A network is provided for accelerating data communication, wherein the network contains: at least one client communication device for originating a data request for obtaining the data from a data server; at least one agent communication device which is assigned to the data server for receiving the data request from the client communication device, wherein the agent keeps track of which client communication devices have received responses to data requests from the assigned data server; at least one peer communication device for storing portions of data received in response to the data request by the at least one client communication device, wherein the portions of data may be transmitted to the at least one client communication device upon request by the client communication device; and at least one acceleration server for deciding which agent communication device is to be assigned to which data server and providing this information to the at least one client communication device.

[0017] The present system and method also provides a communication device within a network, wherein the communication device contains: a memory; and a processor configured by the memory to perform the steps of: originating a data request for obtaining data from a data server; being assigned to a data server, referred to as an assigned data server; receiving a data request from a separate device within the network, and keeping track of which client communication devices within the network have received responses to data requests from the assigned data server; and storing portions of data received in response to the originated data request, wherein the portions of data may be transmitted to communication device upon request by the communication device.

[0018] Other systems, methods, features, and advantages of the present invention will be or become apparent to one with skill in the art upon examination of the following drawings and detailed description. It is intended that all such additional systems, methods, features, and advantages be included within this description, be within the scope of the present invention, and be protected by the accompanying claims.

BRIEF DESCRIPTION OF THE DRAWINGS

[0019] Many aspects of the invention can be better understood with reference to the following drawings. The components in the drawings are not necessarily to scale, emphasis instead being placed upon clearly illustrating the principles of the present invention. Moreover, in the drawings, like reference numerals designate corresponding parts throughout the several views.

[0020] FIG. 1 is a schematic diagram providing a prior art example of use of a proxy within a network.

[0021] FIG. 2 is a schematic diagram providing a prior art example of a peer-to-peer file transfer network.

[0022] FIG. 3 is a schematic diagram providing an example of a communication network in accordance with the present invention.

[0023] FIG. 4 is a schematic diagram further illustrating a communication device of the communication network of FIG. 3.

[0024] FIG. 5 is a schematic diagram further illustrating the memory of FIG. 4.

[0025] FIG. 6 is a schematic diagram further illustrating elements of the acceleration application of FIG. 5, as well as communication paths of the acceleration application.

[0026] FIG. 7 is a chart further illustrating two of the main databases utilized within the communication network.

[0027] FIG. 8 is a flowchart illustrating operation of the acceleration system initializer module.

[0028] FIG. 9 is a flowchart further illustrating communication between different elements of the communication network.

[0029] FIG. 10 is a flowchart continuing the flowchart of FIG. 9 and focused on agent response to the HTTP request.

[0030] FIG. 11 is a flowchart continuing the flowchart of FIG. 10, which illustrates actions taken upon receipt of the list of peers, or single peer listing, from the agent.

[0031] FIG. 12 is a flowchart illustrating steps taken by an agent, client, or peer to determine whether a certain HTTP request is still valid.

[0032] FIG. 13 is a flowchart outlining operation of the acceleration server.

[0033] FIG. 14 is a flowchart further illustrating TCPIP acceleration in accordance with an alternative embodiment of the invention.

[0034] FIG. 15 is a flowchart further illustrating TCPIP acceleration in accordance with an alternative embodiment of the invention, detailing the communication between the client and the TCPIP server (read and write commands) after the connect phase has completed successfully.

DETAILED DESCRIPTION

[0035] The present system and method provides for faster and more efficient data communication within a communication network. An example of such a communication network 100 is provided by the schematic diagram of FIG. 3. The network 100 of FIG. 3 contains multiple communication devices. Due to functionality provided by software stored within each communication device, which may be the same in each communication device, each communication device may serve as a client, peer, or agent, depending upon requirements of the network 100, as is described in detail herein. It should be noted that a detailed description of a communication device is provided with regard to the description of FIG. 4.

[0036] Returning to FIG. 3, the exemplary embodiment of the network 100 illustrates that one of the communication devices is functioning as a client 102. The client 102 is capable of communication with one or more peers 112, 114, 116 and one or more agents 122. For exemplary purposes, the network contains three peers and one agent, although it is noted that a client can communicate with any number of agents and peers.

[0037] The communication network 100 also contains a Web server 152. The Web server 152 is the server from which the client 102 is requesting information and may be, for example, a typical HTTP server, such as those being used to deliver content on any of the many such servers on the Internet. It should be noted that the server 152 is not limited to being an HTTP server. In fact, if a different communication protocol is used within the communication network, the server may be a server capable of handling a different protocol. It should also be noted that while the present description refers to the use of HTTP, the present invention may relate to any other communication protocol and HTTP is not intended to be a limitation to the present invention.

[0038] The communication network 100 further contains an acceleration server 162 having an acceleration server storage device 164. As is described in more detail herein, the acceleration server storage device 164 has contained therein an acceleration server database. The acceleration server database stores Internet protocol (IP) addresses of communication devices within the communication network 100 having acceleration software stored therein. Specifically, the acceleration server database contains stored therein a list of communication devices having acceleration software stored therein that are currently online within the communication network 100. For each such agent, the acceleration server assigns a list of IP addresses.

[0039] In the communication network 100 of FIG. 3, the application in the client 102 is requesting information from the Web server 152, which is why the software within the communication device designated this communication device to work as a client. In addition, since the agent 122 receives the request from the client 102 as the communication device closest to the Web server 152, functionality of

the agent 122, as provided by the software of the agent 122, designates this communication device to work as an agent. It should be noted, that in accordance with an alternative embodiment of the invention, the agent need not be the communication device that is closest to the Web server. Instead, a different communication device may be selected to be the agent.

[0040] Since the peers 112, 114, 116 contain at least portions of the information sought by the client 102 from the Web server 152, functionality of the peers 112, 114, 116, as provided by the software of the peers 112, 114, 116, designates these communication devices to work as peers. It should be noted that the process of designating clients, agents, and peers is described in detail herein. It should also be noted that the number of clients, agents, peers, acceleration servers, Web servers, and other components of the communication network 100 may differ from the number illustrated by FIG. 3. In fact, the number of clients, agents, peers, acceleration servers, Web servers, and other components of the communication network 100 are not intended to be limited by the current description.

[0041] Prior to describing functionality performed within a communication network 100, the following further describes a communication device 200, in accordance with a first exemplary embodiment of the invention. FIG. 4 is a schematic diagram further illustrating a communication device 200 of the communication network 100, which contains general components of a computer. As previously mentioned, it should be noted that the communication device 200 of FIG. 4 may serve as a client, agent, or peer.

[0042] Generally, in terms of hardware architecture, as shown in FIG. 4, the communication device 200 includes a processor 202, memory 210, at least one storage device 208, and one or more input and/or output (I/O) devices 240 (or peripherals) that are communicatively coupled via a local interface 250. The local interface 250 can be, for example but not limited to, one or more buses or other wired or wireless connections, as is known in the art. The local interface 250 may have additional elements, which are omitted for simplicity, such as controllers, buffers (caches), drivers, repeaters, and receivers, to enable communications. Further, the local interface 250 may include address, control, and/or data connections to enable appropriate communications among the aforementioned components.

[0043] The processor 202 is a hardware device for executing software, particularly that stored in the memory 210. The processor 52 can be any custom made or commercially available processor, a central processing unit (CPU), an auxiliary processor among several processors associated with the communication device 200, a semiconductor based microprocessor (in the form of a microchip or chip set), a macroprocessor, or generally any device for executing software instructions.

[0044] The memory 210, which is further illustrated and described by the description of FIG. 5, can include any one or combination of volatile memory elements (e.g., random access memory (RAM, such as DRAM, SRAM, SDRAM, etc.)) and nonvolatile memory elements (e.g., ROM, hard drive, tape, CDROM, etc.). Moreover, the memory 210 may incorporate electronic, magnetic, optical, and/or other types of storage media. Note that the memory 210 can have a distributed architecture, where various components are situated remote from one another, but can be accessed by the processor 202.

[0045] The software **212** located within the memory **210** may include one or more separate programs, each of which contains an ordered listing of executable instructions for implementing logical functions of the communication device **200**, as described below. In the example of FIG. 4, the software **212** in the memory **210** at least contains an acceleration application **220** and an Internet browser **214**. In addition, the memory **210** may contain an operating system (O/S) **230**. The operating system **230** essentially controls the execution of computer programs and provides scheduling, input-output control, file and data management, memory management, and communication control and related services. It should be noted that, in addition to the acceleration application **220**, Internet browser **214**, and operating system **230**, the memory **210** may contain other software applications.

[0046] While the present description refers to a request from the client originating from an Internet browser, the present invention is not limited to requests originating from Internet browsers. Instead, a request may originate from an email program or any other program that would be used to request data that is stored on a Web server, or other server holding data that is requested by the client device.

[0047] Functionality of the communication device **200** may be provided by a source program, executable program (object code), script, or any other entity containing a set of instructions to be performed. When a source program, then the program needs to be translated via a compiler, assembler, interpreter, or the like, which may or may not be included within the memory **210**, so as to operate properly in connection with the operating system **230**. Furthermore, functionality of the communication device **200** can be written as (a) an object oriented programming language, which has classes of data and methods, or (b) a procedure programming language, which has routines, subroutines, and/or functions.

[0048] The I/O devices **240** may include input devices, for example but not limited to, a keyboard, mouse, scanner, microphone, etc. Furthermore, the I/O devices **240** may also include output devices, for example but not limited to, a printer, display, etc. Finally, the I/O devices **240** may further include devices that communicate via both inputs and outputs, for instance but not limited to, a modulator/demodulator (modem; for accessing another device, system, or network), a radio frequency (RF) or other transceiver, a telephonic interface, a bridge, a router, etc.

[0049] When the communication device **200** is in operation, the processor **202** is configured to execute the software **212** stored within the memory **210**, to communicate data to and from the memory **210**, and to generally control operations of the communication device **200** pursuant to the software **212**. The software **212** and the O/S **230**, in whole or in part, but typically the latter, are read by the processor **202**, perhaps buffered within the processor **202**, and then executed.

[0050] When functionality of the communication device **200** is implemented in software, as is shown in FIG. 4, it should be noted that the functionality can be stored on any computer readable medium for use by or in connection with any computer related system or method. In the context of this document, a computer readable medium is an electronic, magnetic, optical, or other physical device or means that can contain or store a computer program for use by or in connection with a computer related system or method. The

functionality of the communication device **200** can be embodied in any computer-readable medium for use by or in connection with an instruction execution system, apparatus, or device, such as a computer-based system, processor-containing system, or other system that can fetch the instructions from the instruction execution system, apparatus, or device and execute the instructions. In the context of this document, a “computer-readable medium” can be any means that can store, communicate, propagate, or transport the program for use by or in connection with the instruction execution system, apparatus, or device.

[0051] The computer readable medium can be, for example but not limited to, an electronic, magnetic, optical, electromagnetic, infrared, or semiconductor system, apparatus, device, or propagation medium. More specific examples (a non-exhaustive list) of the computer-readable medium would include the following: an electrical connection (electronic) having one or more wires, a portable computer diskette (magnetic), a random access memory (RAM) (electronic), a read-only memory (ROM) (electronic), an erasable programmable read-only memory (EPROM, EEPROM, or Flash memory) (electronic), an optical fiber (optical), and a portable compact disc read-only memory (CDROM) (optical). Note that the computer-readable medium could even be paper or another suitable medium upon which the program is printed, as the program can be electronically captured, via for instance optical scanning of the paper or other medium, then compiled, interpreted or otherwise processed in a suitable manner if necessary, and then stored in a computer memory.

[0052] In an alternative embodiment, where the functionality of the communication device **200** is implemented in hardware, the functionality can be implemented with any or a combination of the following technologies, which are each well known in the art: a discrete logic circuit(s) having logic gates for implementing logic functions upon data signals, an application specific integrated circuit (ASIC) having appropriate combinational logic gates, a programmable gate array (s) (PGA), a field programmable gate array (FPGA), etc.

[0053] The at least one storage device **208** of the communication device **200** may be one of many different categories of storage device. As is described in more detail herein, the storage device **208** may include a configuration database **280** and a cache database **282**. Alternatively, the configuration database **280** and cache database **282** may be located on different storage devices that are in communication with the communication device **200**. The description that follows assumes that the configuration database **280** and cache database **282** are located on the same storage device, however, it should be noted that the present invention is not intended to be limited to this configuration.

[0054] The configuration database **280** stores configuration data that is common to all elements of the communication network **100** and is used to provide set up and synchronization information to different modules of the acceleration application **220** stored within the memory **210**, as is described in further detail herein. The cache database **282** stores responses to HTTP requests that the communication device **200** has dispatched, either for its own consumption or on behalf of other elements of the communication network **100**. As is explained in additional detail herein, the responses to HTTP requests are stored within the cache database **282** for future use by this communication device **200**, or for other communication devices within the

communication network 100 that need to retrieve this information and will use this communication device as either a peer or an agent.

[0055] In addition to the abovementioned, as is explained in further detail herein, the cache database 282 has stored therein a list of URLs that the communication device is aware of (i.e., has seen requests for). For each URL, the cache database 282 has stored therein the URL itself, HTTP headers returned by the Web Server for this URL, when the last time was that the contents of this URL was loaded directly from the Web Server, when the contents of the URL had last changed on the Web Server, as well as a list of chunks that contain the contents of this URL, and the chunks of data themselves. Chunks in the present description are defined as equally sized pieces of data that together form the whole content of the URL. It should be noted that while the present description provides for chunks being equally sized pieces of data, in accordance with an alternative embodiment of the invention, the chunks may instead be of different size.

[0056] FIG. 5 is a schematic diagram further illustrating the memory 210 of FIG. 4. As shown by FIG. 5, the memory 210 may be separated into two basic levels, namely, an operating system level 260 and an application level 270. The operating system level 260 contains the operating system 230, wherein the operating system 230 further contains at least one device driver 262 and at least one communication stack 264. The device drivers 262 are software modules that are responsible for the basic operating commands for various hardware devices of the communication device 200, such as the processor 202, the storage device 208 and the I/O devices 240. In addition, the communication stacks 264 provide applications of the communication device 200 with a means of communicating within the network 100 by implementing various standard communication protocols.

[0057] The application level 270 includes any application that is running on the communication device 200. As a result, the application level 270 includes the Internet browser 214, which is used to view information that is located on remote Web servers, the acceleration application 220, as described in more detail below, and any other applications 216 stored on the communication device 200.

[0058] As is explained in additional detail below, the acceleration application 220 intercepts the requests being made by applications of the communication device (client) that use the Internet, in order to modify the requests and route the requests through the communication network. There are various methods that may be used to intercept such requests. One such method is to create an intermediate driver 272, which is also located within the memory 210, that attaches itself to all communication applications, intercepts outgoing requests of the communication applications of the communication device 200, such as the Internet browser 214, and routes the requests to the acceleration application 220. Once the acceleration application 220 modifies the requests, routes the requests to other system elements on the communication network 100, and receives replies from other system elements of the communication network 100, the acceleration application 220 returns the replies to the intermediate driver 272, which provides the replies back to the requesting communication application.

[0059] FIG. 6 is a schematic diagram further illustrating elements of the acceleration application 220, as well as communication paths of the acceleration application 220.

The acceleration application 220 contains an acceleration system initializer module 222, which is called when the acceleration application 220 is started. The acceleration system initializer module 222 is capable of initializing all elements of the communication device 200. The acceleration application 220 also contains three separate modules that run in parallel, namely, a client module 224, a peer module 226, and an agent module 228, each of which comes into play according to the specific role that the communication device 200 is partaking in the communication network 100 at a given time. The role of each module is further described herein.

[0060] The client module 224 provides functionality required when the communication device 200 is requesting information from the Web server 152, such as, for example, but not limited to, Web pages, data, video, or audio. The client module 224 causes the communication device 200 having the client module 224 therein to intercept the information request and pass the information request on to other elements of the communication network 100, such as, servers, agents or peers. This process is further described in detail herein.

[0061] The peer module 226 provides functionality required by the communication device 200 when answering other clients within the communication network 100 and providing the other clients with information that they request, which this communication device 200, having this peer module 226 therein, has already downloaded at a separate time. This process is further described in detail herein.

[0062] The agent module 228 provides functionality required when other communication devices of the communication network 100 acting as clients query this communication device 200, having this agent module 228 therein, as an agent, to obtain a list of peers within the communication network 100 that contain requested information. This process is further described in detail herein.

[0063] The acceleration application 220 interacts with both the configuration database 280 and the cache database 282 of the storage device 208. As previously mentioned herein, the configuration database 280 stores configuration data that may be common to all communication devices of the communication network 100 and is used to provide setup and synchronization information to different modules 222, 224, 226, 228 of the acceleration application 220 stored within the memory 210.

[0064] The cache database 282 stores responses to information requests, such as, for example, HTTP requests, that the communication device 200 has dispatched, either for its own consumption or on behalf of other elements of the communication network 100. The responses to HTTP requests are stored within the cache database 282 for future use by this communication device 200, or for other communication devices within the communication network 100 that need to retrieve this same information and will use this communication device 200 as either a peer or an agent. This process is described in detail herein.

[0065] Information stored within the cache database 282 may include any information associated with a request sent by the client. As an example, such information may include, metadata and actual requested data. For example, for an HTTP request for a video, the metadata may include the version of the Web server answering the request from the client and the data would be the requested video itself. In a

situation where there is no more room for storage in the cache database, the software of the associated communication device may cause the communication device to erase previous data stored in order to clear room for the new data to store in the cache database. As an example, such previous data may include data that is most likely not to be used again. Such data may be old data or data that is known to no longer be valid. The communication device may choose to erase the least relevant data, according to any of several methods that are well known in the art.

[0066] FIG. 7 is a chart further illustrating two of the main databases utilized within the communication network **100**, namely, the acceleration server database **164** and the cache database **282**. As previously mentioned, the acceleration server database **164** stores IP addresses of communication devices located within the communication network **100**, which have acceleration software stored therein. Specifically, the acceleration server database **164** contains stored therein a list of communication devices having acceleration software stored therein that are currently online within the communication network **100**. The acceleration server assigns a list of IP addresses to each communication device functioning as an agent. Each communication device will be the agent for any Web servers whose IP address is in the range 'owned' by that communication device. As an example, when a first ever communication device goes online, namely, the first communication device as described herein having the acceleration application **220** therein, the acceleration server assigns all IP addresses in the world to this communication device, and this communication device will be the agent for any Web server. When a second communication device goes online it will share the IP address list with the first communication device, so that each of the communication devices will be responsible for a different part of the world wide web servers.

[0067] The cache database **282** of the communication device **200** has stored therein a list of URLs **286** of which the communication device **200** is aware. The communication device **200** becomes aware of a URL each time that the communication device **200** receives a request for information located at a specific URL. As shown by FIG. 7, for each URL **288** within the list of URLs **286**, the cache database **282** stores: the URL itself **290**; HTTP headers **292** returned by the Web Server **152** for this URL; when the last time **294** was that the contents of this URL were loaded directly from the Web Server **152**; when the contents of the URL last changed **296** on the Web Server **152**; and a list of chunks **298** that contain the contents of this URL, and the content of the chunk. As previously mentioned, chunks, in the present description, are defined as equally sized pieces of data that together form the entire content of the URL, namely, the entire content whose location is described by the URL. As a non-limiting example, a chunk size of, for example, 16 KB can be used, so that any HTTP response will be split up into chunks of 16 KB. In accordance with an alternative embodiment of the invention, if the last chunk of the response is not large enough to fill the designated chunk size, such as 16 KB for the present example, the remaining portion of the chunk will be left empty.

[0068] For each such chunk **300**, the cache database **282** includes the checksum of the chunk **302**, the data of the chunk **304** itself, and a list of peers **306** that most likely have the data for this chunk. As is described in additional detail herein, the data for the chunk may be used by other clients

within the communication network **100** when other communication devices of the communication network **100** serve as peers to the clients, from which to download the chunk data.

[0069] For each chunk, a checksum is calculated and stored along side of the chunk itself. The checksum may be calculated in any of numerous ways known to those in the art. The purpose of having the checksum is to be able to identify data uniquely, whereas the checksum is the "key" to the data, where the data is the chunk. As an example, a client may want to load the contents of a URL, resulting in the agent that is servicing this request sending the checksums of the chunks to the client, along with the peers that store these chunks. It is to be noted that there could be a different peer for every different chunk. The client then communicates with each such peer, and provides the checksum of the chunk that it would like the peer to transmit back to the client. The peer looks up the checksum (the key) in its cache database, and provides back the chunk (data) that corresponds to this checksum (the key). As shown by FIG. 7, for each peer **308** within the list of peers **306**, the cache database **282** includes the peer IP address **310**, as well as the connection status **312** of the peer, which represents whether the peer **308** is online or not.

[0070] In accordance with one embodiment of the invention, the cache database **282** may be indexed by URL and by Checksum. Having the cache database indexed in this manner is beneficial due to the following reason. When the agent is using the cache database, the agent receives a request from a client for the URL that the client is looking for. In such a case the agent needs the cache database to be indexed by the URL, to assist in finding a list of corresponding peers that have the chunks of this URL. When the peers are using this cache database, the peers obtain a request from the client for a particular checksum, and the peers need the database to be indexed by the checksum so that they can quickly find the correct chunk. Of course, as would be understood by one having ordinary skill in the art, the cache database may instead be indexed in any other manner.

[0071] Having described components of the communication network **100**, the following further describes how such components interact and individually function. FIG. 8 is a flowchart **300** illustrating operation of the acceleration system initializer module **222** (hereafter referred to as the initializer **222** for purposes of brevity). It should be noted that any process descriptions or blocks in flowcharts should be understood as representing modules, segments, portions of code, or steps that include one or more instructions for implementing specific logical functions in the process, and alternative implementations are included within the scope of the present invention in which functions may be executed out of order from that shown or discussed, including substantially concurrently or in reverse order, depending on the functionality involved, as would be understood by those reasonably skilled in the art of the present invention.

[0072] The initializer **222** is the first element of the communication device **200** to operate as the communication device **200** starts up (block **302**). As the initializer **222** starts, it first communicates with the acceleration server **162** to sign up with the acceleration server **162**. This is performed by providing the acceleration server **162** with the hostname, and all IP addresses and media access control (MAC) addresses of the interfaces on the communication device **200** having the initializer **222** thereon.

[0073] In accordance with an alternative embodiment of the invention, as shown by block 304, the initializer 222 checks with the acceleration server 162 whether a more updated version of the acceleration application software is available. This may be performed by any one of many known methods, such as, but not limited to, by providing the version number of the acceleration application software to the acceleration server 162. The message received back from the acceleration server 162 indicates whether there is a newer version of the acceleration application software or not. If a newer version of the acceleration application software exists, the initializer 222 downloads the latest version of the acceleration application software from the acceleration server 162, or from a different location, and installs the latest version on the communication device 200. In addition to the abovementioned, the initializer 222 may also schedule additional version checks for every set period of time thereafter. As an example, the initializer 222 may check for system updates every two days.

[0074] As shown by block 306, the initializer 222 then redirects outgoing network traffic from the communication device 200 to flow through the acceleration application 162. As previously mentioned, one way to redirect the outgoing network traffic is to insert an intermediate driver 212 that intercepts and redirects the traffic. It should be noted that there are many other ways to implement this redirection, which are well known to those having ordinary skill in the art.

[0075] As shown by block 308, the initializer 222 then launches the client module 224 of the communication device 200, and configures the client module 224 of the communication device 200 to intercept to all outgoing network communications of the communication device 200 and route the outgoing network communications to the client module 224, from the intermediate driver 272 or other routing method implemented. This is performed so that the client module 224 is able to receive all network traffic coming from the network applications, modify the network traffic if necessary, and re-route the traffic. As is known by those having ordinary skill in the art, in order to re-route the traffic, the traffic needs to be modified, as an example, to change the destination of requests.

[0076] As shown by block 310, the initializer 222 then launches the agent module 228 and the peer module 226 to run on the communication device 200. The agent module 228 and peer module 226 listen on pre-determined ports of the communication device 200, so that incoming network traffic on these ports gets routed to the agent module 228 and peer module 226. As is explained in further detail herein, the abovementioned enables the communication device 200 to function as an agent and as a peer for other communication devices within the communication network 100, as needed.

[0077] FIG. 9 is a flowchart 350 further illustrating communication between different elements of the communication network 100, in accordance with the present system and method for providing faster and more efficient data communication.

[0078] As shown by block 352, an application running on the client 200 initiates a request for a resource on a network. Such a request may be, for example, "GET http://www.aol.com/index.html HTTP/1.1". The request may come from an Internet browser 214 located on the client 200, where the Internet browser 214 is loading a page from the Internet, an

application that wants to download information from the Internet, fetch or send email, or any other network communication request.

[0079] Through the intermediate driver 272, or other such mechanism as may be implemented that is re-routing the communication to the client module 224 of the client 200, the resource request is intercepted by the client module 224 that is running on the client 200 (block 354). The client module 224 then looks up the IP address of the server 152 that is the target of the resource request (e.g., the IP address of the Web server that is the host of www.aol.com in the example above), and sends this IP address to the acceleration server 162 (block 356) in order to obtain a list of communication devices that the client 200 can use as agents (hereafter referred to as agents). It should be noted that the process of performing an IP lookup for a server is known by one having ordinary skill in the art, and therefore is not described further herein.

[0080] In response to receiving the IP address of the server 152, the acceleration server 162 prepares a list of agents that may be suitable to handle the request from this IP address (block 358). The size of the list can differ based on implementation. For exemplary purposes, the following provides an example where a list of five agents is prepared by the acceleration server 162. The list of agents is created by the acceleration server 162 by finding the communication devices of the communication network 100 that are currently online, and whose IP address is numerically close to the IP of the destination Web server 152. A further description of the abovementioned process is described here in.

[0081] As shown by block 360, the client module 224 then sends the original request (e.g., "GET http://www.aol.com/index.html HTTP/1.1") to all the agents in the list received from the acceleration server 162 in order to find out which of the agents in the list is best suited to be the one agent that will assist with this request.

[0082] It should be noted that, in accordance with an alternative embodiment of the invention, the communication device 200 may be connected to a device that is actually requesting data. In such an alternative embodiment, the communication device would be a modular device connected to a requesting device, where the requesting device, such as, for example, a personal data assistant (PDA) or other device, would request data, and the communication device connected thereto, either through a physical connection, wireless connection, or any other connection, would receive the data request and function as described herein. In addition, as previously mentioned, it should be noted that the HTTP request may be replaced by any request for resources on the Web.

[0083] FIG. 10 is a flowchart continuing the flowchart 380 of FIG. 9 and focused on agent response to the request. As shown by block 382, upon receiving the request from the client 200, each agent that received the request from the client responds to the client 200 with whether it has information regarding the request, which can help the client to download the requested information from peers in the network. Specifically, each agent responds with whether the agent has seen a previous request for this resource that has been fulfilled. In such a case, the agent may then provide the client with the list of peers and checksums of the chunks that each of them have.

[0084] As shown by block 384, the client then decides which of the agents in the list to use as its agent for this

particular information request. To determine which agent in the list to use as its agent for the particular information request, the client may consider multiple factors, such as, for example, factoring the speed of the reply by each agent and whether that agent does or does not have the information required. There are multiple ways to implement this agent selection, one practical way being to start a timer of a small window of time, such as, for example, 5 ms, after receiving the first response from the agents, and after the small window, choosing from the list of agents that responded, the agent that has the information about the request, or in the case that none of the agents responded, to choose the first agent from the list received from the acceleration server 162.

[0085] As shown by block 386, after selecting an agent, the client notifies the selected agent that it is going to use it for this request, and notifies the other agents that they will not be used for this request. The client then sends the selected agent a request for the first five chunks of data of the original information request (block 388). By specifying to the selected agent the requested chunks by their order in the full response, the client receives the peer list and checksums of the requested chunks from the selected agent. As an example, for the first five chunks the client will ask the selected agent for chunks one through five, and for the fourth batch of five chunks the client will ask the agent for chunks sixteen through twenty. As previously mentioned, additional or fewer chunks may be requested at a single time.

[0086] As shown by block 390, after receiving the request from the client, the selected agent determines whether it has information regarding the requested chunks of data by looking up the request in its cache database and determining if the selected agent has stored therein information regarding peers of the communication network that have stored the requested data of the request, or whether the selected agent itself has the requested data of the request stored in its memory. In addition to determining if the selected agent contains an entry for this request in its database, the selected agent may also determine if this information is still valid. Specifically, the selected agent determines whether the data that is stored within the memory of the selected agent or the memory of the peers, still mirrors the information that would have been received from the server itself for this request. A further description of the process utilized by the selected agent to determine if the information is still valid, is described in detail herein.

[0087] As shown by block 392, if the information (requested data of the request) exists and is still valid, then the agent prepares a response to the client, which includes for each of the chunks: (i) the checksum of the chunk; (ii) a list of peers that according to the database of the selected agent contains these chunks; and (iii) if these are the first five chunks of the information, then the selected agent also provides the specific protocol's headers that would have been received from the server, had the initial request from the client been made directly to the server.

[0088] As shown by block 394, the list of peers for each chunk is sorted by geographical proximity to the requesting client. In accordance with the present example, only the five closest peers are kept in the list for every chunk, and the rest of the peers are discarded from this list. As shown by block 396, the prepared response, namely, the list of closest peers, is sent back to the client. It should be noted that, if this were

the last set of chunks to be provided for this request, then it would be beneficial to include information about this to the client.

[0089] If the selected agent discovers that it does not have information about this request, or if the selected agent discovers that the information it has is no longer valid, the selected agent needs to load the information directly from the server in order to be able to provide an answer to the requesting client. As shown by block 400, the selected agent then sends the request directly to the server. The selected agent then stores the information it receives from the server (both the headers of the request, as well as chunks of the response itself) in its database, for this particular response to the client, as well as for future use to other clients that may request this data (block 402). The selected agent then prepares a response (list) for the client, where the response includes the protocol headers (if these are the first five chunks), and the checksums of the five chunks, and provides itself as the only peer for these chunks (block 404). This list is then sent back to the client (block 406).

[0090] FIG. 11 is a flowchart 420 continuing the flowchart of FIG. 10, which illustrates actions taken upon receipt of the list of peers, or single peer listing, from the agent. As shown by block 422, the client receives the response from the agent (including the list of chunks and their corresponding data, including peers and other information previously mentioned) and, for each of the five chunks, the client sends a request to each of the peers listed for the chunk to download the chunk. The chunk request that the client sends to each of the peers is the checksum of the data that the client seeks to receive, which is the key (identifier) of the chunk.

[0091] As shown by block 424, the peers then respond regarding whether they still have the data of the chunk. As an example, some of the peers may not currently be online, some may be online but may have discarded the relevant information, and some may still have the relevant information, namely, the chunk. As shown by block 426, the client then selects the quickest peer that responds with a positive answer regarding the requested information, the client lets that peer know that it is chosen to provide the client with the chunk, and the client notifies the other peers that they are not chosen.

[0092] As shown by block 428, the chosen peer then sends the chunk to the client. It should be noted that if no peers answer the request of the client, the client goes back to the agent noting that the peers were all negative, and the agent either provides a list of 5 other agents, if they exist, or the agent goes on to download the information directly from the Web server as happens in the case where no peers exist as described above.

[0093] The client then stores the chunks in its cache for future use (block 430), when the client may need to provide the chunks to a requesting communication device when acting as a peer for another client that is looking for the same information. As shown by block 432, if some of the chunks were not loaded from any of the peers, the client requests the chunks again from the agent in a next round of requests, flagging these chunks as chunks that were not loadable from the client list of peers. In this situation, the agent will load the data directly from the server and provide it back to the client.

[0094] The client then acknowledges to the agent which of the chunks it received properly (block 434). The agent then looks up these chunks in the database of the agent, and adds

the client to the list of peers for these chunks, specifically, since this client is now storing these chunks, and can provide these chunks to other clients that turn to it as a peer (block 436).

[0095] As shown by block 438, the client then passes the data on to the Web browser or other application of the client that made the original request, for it to use as it had originally intended. The client then checks whether all of the chunks for this request were received (block 440), by checking the flag set by the agent. Specifically, when the agent is providing the list of the last 5 chunks, the agent includes that information as part of its reply to the client, which is referred to herein as a flag. This information is what enables the client to know that all information has been received for a particular resource request.

[0096] If the last received chunks were not the last chunks for this request, the processing flow of the client continues by returning to the functionality of block 384 of FIG. 10. but instead sending the chosen agent a request for the next five chunks of data of the original information request. Alternatively, if all chunks for this request were received, the request is complete, and the flow starts again at block 352 of FIG. 9.

[0097] FIG. 12 is a flowchart 500 illustrating steps taken by an agent, client, or peer to determine whether a certain HTTP request is still valid. Specifically, the following provides an example of how the agent, client, or peer can determine whether particular data that is stored within the memory of the agent, or the memory of a peer or client, still mirrors the information that is currently on the Web server. As shown by block 502, the HTTP request is looked up in the cache database of the agent, client or peer that is checking the validity of the HTTP request. As an example, the HTTP protocol, defined by RFC 2616, outlines specific methods that Web servers can define within the HTTP headers signifying the validity of certain data, such as, but not limited to, by using HTTP header information such as “max age” to indicate how long this data may be cached before becoming invalid, “no cache” to indicate that the data may never be cached, and using other information.

[0098] As shown by block 504, these standard methods of validation are tested on the HTTP request information in question. As shown by block 506, a determination is made whether the requested information that is stored is valid or not. If the requested information is valid, a “VALID” response is returned (block 508). Alternatively, if the requested information is not valid, an HTTP conditional request is sent to the relevant Web server, to determine if the data stored for this request is still valid (block 510). If the data stored for this request is still valid, a “VALID” response is returned (block 508). Alternatively, if the data stored for this request is not valid, an “INVALID” response is returned (block 514). It should be noted, that the abovementioned description with regard to FIG. 12 is an explanation of how to check if HTTP information is still valid. There are similar methods of determining validity for any other protocol, which may be utilized, and which those having ordinary skill in the art would appreciate and understand.

[0099] FIG. 13 is a flowchart 550 outlining operation of the acceleration server, whose main responsibility in the present system and method is to provide clients with information regarding which agents serve which requests, and to keep the network elements all up to date with the latest software updates. As shown by block 552, the acceleration

server sends “keep alive” signals to the network elements, and keeps track within its database as to which network elements are online. As shown by block 554, the acceleration server continues to wait for a client request and continues to determine if one is received.

[0100] Once a request is received, the acceleration server tests the type of request received (block 556). If the client request is to sign up the client within the network, an event that happens every time that the client starts running on its host machine, then that client is added to the list of agents stored on the acceleration server, sorted by the IP address of the client (block 558).

[0101] If the request is to find an agent to use for a particular request, the acceleration server creates a new agent list, which is empty (block 560). The acceleration server then searches the agent database for the next 5 active agents whose IP address is closest to the IP address of the server who is targeted in the request (block 562). In this context, 192.166.3.103 is closer to 192.166.3.212 than to 192.167.3.104. The acceleration server then sends this agent list to the client (block 564).

[0102] If instead, the request is to check the version of the latest acceleration software then the acceleration server sends that network element (client, peer or agent) the version number of the latest existing acceleration software version, and a URL from where to download the new version, for the case that the element needs to upgrade to the new version (block 566).

[0103] While the abovementioned example is focused on HTTP requests for data, as previously mentioned, other protocol requests are equally capable of being handled by the present system and method. As an example, in separate embodiments the acceleration method described may accelerate any communication protocol at any OSI layer (SMTP, DNS, UDP, ETHERNET, etc.). In the following alternative embodiment, it is illustrated how the acceleration method may accelerate TCP/IP. As is known by those having ordinary skill in the art, TCP/IP is a relatively low-level protocol, as opposed to HTTP, which is a high level protocol. For purposes of illustration of TCP/IP communication, reference may be made to FIG. 3, wherein the Web server is a TCP/IP server.

[0104] In TCP/IP there are three communication commands that are of particular interest, namely, connect, write, and read. Connect is a command issued by an application in the communication device that is initiating the communication to instruct the TCP/IP stack to connect to a remote communication device. The connect message includes the IP address of the communication device, and the port number to connect to. An application uses the write command to instruct the TCP/IP stack to send a message (i.e., data) to a communication device to which it is connected. In addition, an application uses the read command to ask the TCP/IP stack to provide the message that was sent from the remote communication device to which it is connected. A communication session typically exists of a connect, followed by a read and write on both sides.

[0105] FIG. 14 is a flowchart 600 further illustrating TCP/IP acceleration in accordance with this alternative embodiment of the invention. As shown by blocks 601 and 602 when an application of the communication device makes a request to the communications stack to connect with the TCP/IP server, that communication is intercepted by the acceleration application.

[0106] To find an agent, upon receiving that connect message from the communication device application, which includes the IP address of the TCPIP server and the port to connect to, the acceleration application in the client makes a request to the acceleration server to find out who the agent for the communication with the TCPIP server is. This step is performed in a similar manner to that described with regard to the main HTTP embodiment of the invention (block 604). As shown by block 606, the server then provides the client with a list of agents, for example, a primary agent and four others.

[0107] To establish a connection, as shown by block 608, the client issues a TCPIP connect with the primary agent or one of the other agents if the primary agent does not succeed, to create a connection with the agent. The client then sends to the agent the IP address of the TCPIP server and connection port that were provided by the communication device application (block 610). As shown by block 612, that agent in turn issues a TCPIP connect to the TCPIP server to the port it received from the client, to create a connection with the agent.

[0108] FIG. 15 is a flowchart 800 further illustrating TCPIP acceleration in accordance with this alternative embodiment of the invention, detailing the communication between the client and the TCPIP server (read and write commands) after the connect phase has completed successfully.

[0109] As shown by block 802, if the network application within the client wants to send a message to the TCPIP server, the network application within the client writes the message to the TCPIP stack in the operating system of the client. This WRITE command is received by the acceleration application of the client and handled in the manner described below. If the TCPIP server wants to send a message to the client, the TCPIP server writes the message to the TCPIP stack of TCPIP operating system, on the connection to the agent, since this agent is where the server received the original connection. This WRITE command is received by the acceleration application of the agent and handled in the manner described below.

[0110] When the acceleration application of the client receives a message from the network application of the client to be sent to the agent, or when the acceleration application of the agent receives a message from the connection to the TCPIP server that is to be sent to the client, the acceleration application proceeds to send the message to the communication device on the other side. For instance, if the client has intercepted the message from the communication application, the client sends the message to the agent, and if it is the agent that intercepted the message from the connection to the TCPIP server, such as the TCPIP server sending a message that is intended for the communication with client, the agent sends the message to the client in the following manner:

[0111] As shown by block 804, the acceleration application breaks up the content of the message to chunks and calculates the corresponding checksums, in the same manner as in the main embodiment described herein. The acceleration application then looks up each checksum in its cache database (block 806). As shown by block 808, the acceleration application checks if the checksum exists in the cache database. Hit does, then, as shown by block 810, the acceleration application prepares a list of peers that have already received the chunk of the checksum in the past (if

any), and adds the communication device of the other side to the list of communication devices that have received this chunk (adds it to the peer list of the checksum in its database), to be provided to other communication devices requesting this information in the future. As shown by block 812, the list of peers is sent to the receiving communication device, which, as shown by block 814 retrieves the chunks from the peers in the list received, in the same manner as in the main embodiment.

[0112] If the checksum does not exist within the cache database of the sending communication device then, as shown by block 820, the acceleration application adds the checksum and chunk to its cache database, sends the chunk to the communication device on the other side, and adds the other communication device to the list of peers for that checksum in its database.

[0113] As shown by block 816, a determination is then made as to whether all chunks have been received. If all chunks have not been received, the process continues on again from block 806.

[0114] Once all data has been received, as shown by block 818, the acceleration application passes the data on to the requester. Specifically, in the client, the acceleration application passes on the complete data to the communication application, and in the agent, the acceleration application passes on the complete data to the requesting TCPIP server.

[0115] It should be emphasized that the above-described embodiments of the present invention are merely possible examples of implementations, merely set forth for a clear understanding of the principles of the invention. Many variations and modifications may be made to the above-described embodiments of the invention without departing substantially from the spirit and principles of the invention. All such modifications and variations are intended to be included herein within the scope of this disclosure and the present invention and protected by the following claims.

1. A non-transitory computer readable medium comprising instructions which, when executed by one or more hardware processors, cause performance of steps comprising:

storing, a group of Internet Protocol (IP) addresses, wherein each of the IP addresses in the group is associated with a respective consumer communication device that is currently online and is addressed in the Internet using a respective IP address;

identifying, a first request;

selecting, in response to the identifying of the first request, an IP address from the group;

sending, over the Internet to a first consumer communication device that is addressed by the selected IP address, the first request, in response to the selecting;

receiving, from the first consumer communication device over the Internet, in response to the sending of the first request, a first content; and

sending, to a second device over the Internet, the received first content,

wherein the first content is identified by a first content identifier, and wherein the first request comprises the first content identifier.

2. The non-transitory computer readable medium according to claim 1, wherein the first content comprises an audio or a video data.

3. The non-transitory computer readable medium according to claim 1, wherein the first content comprises a web-

page, and wherein the first content identifier comprises a Uniform Resource Locator (URL) that identifies the web-page in the Internet.

4. The non-transitory computer readable medium according to claim 1, wherein the identifying comprises receiving, over the Internet, the first content identifier.

5. The non-transitory computer readable medium according to claim 4, wherein the receiving of the first content identifier comprises receiving of the first content identifier from the second device.

6. The non-transitory computer readable medium according to claim 1, wherein the second device comprises a second consumer communication device.

7. The non-transitory computer readable medium according to claim 1, for use with a server that stores the first content, wherein the instructions, when executed by the one or more hardware processors, further cause performance of: receiving, from the server over the Internet, the first request, in response to the sending of the first request, to the first device over the Internet;

sending, by the first device to the server over the Internet, a second request that comprises, or is based on, the received first request, in response to the receiving of the first request; and

receiving, by the first device from the server over the Internet, the first content, in response to sending of the second request.

8. The non-transitory computer readable medium according to claim 7, wherein the server comprises a web-server and wherein the first content comprises a web-page.

9. The non-transitory computer readable medium according to claim 1, wherein the instructions, when executed by the one or more hardware processors, further cause performance of communicating, over the Internet, with each of the consumer communication devices in the group that are addressed over the Internet using the stored IP addresses.

10. The non-transitory computer readable medium according to claim 9, wherein the storing is in response to the communicating.

11. The non-transitory computer readable medium according to claim 9, wherein the communicating comprises receiving a 'keep alive' message, over the Internet, from each of the consumer communication devices.

12. The non-transitory computer readable medium according to claim 1, wherein the instructions, when executed by the one or more hardware processors, further cause performance of establishing, a Transmission Control Protocol (TCP) connection with each of the consumer communication devices that are identified over the Internet using the respective IP addresses in the group.

13. The non-transitory computer readable medium according to claim 1, wherein the instructions, when executed by the one or more hardware processors, further cause performance of:

sending, over the Internet, a 'keep alive' message to at least one of the consumer communication devices that are identified over the Internet using the stored IP addresses; and

waiting, for a response to the 'keep alive' message from the at least one of the consumer communication devices.

14. The non-transitory computer readable medium according to claim 13, wherein the instructions, when

executed by the one or more hardware processors, further cause performance of responsive to not receiving, over the Internet, a response to the 'keep alive' message from the at least one consumer communication device, removing the IP address of the non-responsive at least one consumer communication device from the group of IP addresses.

15. The non-transitory computer readable medium according to claim 1, for use with a criterion, wherein the selecting is according to, or based on, the criterion.

16. The non-transitory computer readable medium according to claim 15, wherein the criterion is based on, or comprises, a numerical closeness to a server IP address.

17. The non-transitory computer readable medium according to claim 15, wherein the criterion is based on, or comprises, a geographical location of the first consumer communication device addressed by the selected IP address.

18. The non-transitory computer readable medium according to claim 15, wherein the criterion is based on, or comprises, a response time when communicating.

19. The non-transitory computer readable medium according to claim 1, for use with a software application stored in the first server and associated with a first version number, and wherein the instructions, when executed by the one or more hardware processors, further cause performance of sending, to the first consumer communication device, a message that comprises the first version number of the software application.

20. The non-transitory computer readable medium according to claim 19, wherein the instructions, when executed by the one or more hardware processors, further cause performance of sending, to the first consumer communication device, the software application.

21. The non-transitory computer readable medium according to claim 1, wherein the instructions, when executed by the one or more hardware processors, further cause performance of communicating over the Internet with the first consumer communication devices based on, according to, or using, TCP/IP protocol or connection.

22. The non-transitory computer readable medium according to claim 1, wherein the instructions, when executed by the one or more hardware processors, further cause performance of communicating over the Internet based on, or according to, an UDP or TCP standard.

23. The non-transitory computer readable medium according to claim 1, wherein the instructions, when executed by the one or more hardware processors, further cause performance of storing, operating, or using, a server operating system.

24. The non-transitory computer readable medium according to claim 1, wherein the first request comprises a Hypertext Transfer Protocol (HTTP) request.

25. The non-transitory computer readable medium according to claim 1, wherein the instructions, when executed by the one or more hardware processors, further cause performance of communicating over the Internet based on, or according to, one out of DNS, FTP, POP3, SMTP, or SQL standards.

* * * * *